



UNIVERSIDAD DEL BÍO-BÍO, CHILE

FACULTAD DE CIENCIAS EMPRESARIALES

Departamento de Sistemas de Información

RENDIMIENTO Y DESEMPEÑO DEL APRENDIZAJE DE DISTINTAS INTELIGENCIAS ARTIFICIALES EN UN ENTORNO DE VIDEOJUEGO OFFLINE

PROYECTO DE TÍTULO
INGENIERÍA EN EJECUCIÓN EN COMPUTACIÓN E INFORMÁTICA
GONZALO IGNACIO FERNANDO CERDA LÓPEZ
DIRIGIDA POR
LUIS EMILIO CABRERA CROT

2026

Resumen

Este trabajo investiga el rendimiento y desempeño del aprendizaje de distintas inteligencias artificiales en un entorno de videojuego ejecutado completamente offline, sin depender de infraestructura en la nube. El problema central abordado es la ausencia de evidencia experimental que cuantifique la viabilidad del aprendizaje de IA en tiempo real bajo restricciones de hardware local, y la necesidad de explorar alternativas que ofrezcan adaptabilidad sin depender de entrenamiento online continuo.

Se desarrolló una plataforma experimental modular basada en el videojuego de turnos Knucklebones, sobre la cual se entrenaron y evaluaron tres paradigmas de aprendizaje: aprendizaje por refuerzo (PPO), neuroevolución (NEAT) y árboles de decisión destilados por *Behavior Cloning*. Para cada paradigma se midieron rigurosamente los datos requeridos, el tiempo de entrenamiento, los recursos computacionales y el desempeño del agente resultante.

El Capítulo 1 presenta la problemática y los objetivos centrados en evaluar la viabilidad del aprendizaje. El Capítulo 2 describe los fundamentos teóricos, incluyendo la distinción entre aprendizaje online pleno y adaptación por selección. El Capítulo 3 revisa el estado del arte con énfasis en casos industriales que evidencian la ausencia de aprendizaje en tiempo real. El Capítulo 4 detalla la arquitectura, el protocolo experimental y los criterios de viabilidad. El Capítulo 5 presenta los resultados: costos de entrenamiento, análisis de brecha de datos, evaluación de sistemas adaptativos y discusión. El Capítulo 6 sintetiza conclusiones y trabajo futuro.

Los resultados demuestran que los tres paradigmas requieren entre 2 y 4 órdenes de magnitud más datos de los que un usuario genera en una sesión real (180–1,800 muestras vs. 400,000–4,590,000 requeridas), lo cual impide el entrenamiento online continuo durante la interacción. Este patrón podría extenderse a otros sistemas con restricciones similares de recursos locales y volumen de interacciones por sesión.

Sin embargo, el entrenamiento offline en hardware doméstico es viable: todos los modelos se entrenaron en menos de 33 minutos por especialista, sin GPU. El sistema NEAT alcanzó un winrate de 0.653 (IC 95 %: [0.645, 0.660]), significativamente superior a DT (0.608) y PPO (0.604). Como alternativas al aprendizaje en tiempo real, se implementaron: (1) un mecanismo de adaptación por selección (KMeans + especialistas pre-entrenados) que opera con latencia de inferencia inferior a 0.1 ms y alcanza rendimiento equivalente al mejor generalista fijo; y (2) la posibilidad de recolectar datos durante las sesiones reales para reentrenamiento posterior, aprovechando datos reales sin el costo del entrenamiento durante la interacción.

Se concluye que la adaptación parcial —mediante perfilado del usuario, selección de políticas pre-entrenadas y recolección de datos para reentrenamiento periódico— constituye la alternativa práctica demostrada frente al entrenamiento online continuo en hardware local, con potencial aplicabilidad a otros dominios que enfrenten restricciones similares.

Palabras Clave — videojuegos, inteligencia artificial, aprendizaje offline, viabilidad del aprendizaje, adaptación por selección, aprendizaje por refuerzo, neuroevolución, árboles de decisión, hardware local

Dedicatoria

A mi familia, por el apoyo constante durante el desarrollo de este trabajo y sobretodo a mi padre, quien estaría orgulloso de verme salir adelante.

Lista de Acrónimos

BC *Behavior Cloning* (Aprendizaje por imitación supervisado).
CPU *Central Processing Unit*.
DT *Decision Tree* (Árbol de Decisión).
FPS *First Person Shooter* (Videojuego en primera persona).
FSM *Finite State Machine*.
GPU *Graphics Processing Unit*.
Gym Interfaz estandarizada de entorno de entrenamiento tipo OpenAI Gym.
HTTP *HyperText Transfer Protocol*.
IA Inteligencia Artificial.
JSON *JavaScript Object Notation*.
JSONL *JavaScript Object Notation Lines*.
ML *Machine Learning*.
NEAT *NeuroEvolution of Augmenting Topologies*.
NPC *Non-Player Character*.
PPO *Proximal Policy Optimization*.
RL *Reinforcement Learning*.
RNG *Random Number Generator*.
SB3 Stable-Baselines3 (biblioteca de RL en Python).
API *Application Programming Interface*.
ANN *Artificial Neural Network*.
WR *Winrate* (Tasa de victoria).

Glosario

Juego Inmutable Principio arquitectónico donde el motor del juego no contiene lógica específica de agentes de inteligencia artificial, limitándose a exponer estado y recibir acciones.

Arquitectura Plug-and-Play Diseño modular que permite intercambiar agentes de IA sin modificar el núcleo del videojuego.

Middleware Capa intermedia encargada de recibir el estado del juego, procesarlo, enrutar la decisión del agente y registrar métricas experimentales.

Función de Transición Función determinista $Ts, a = s'$ que define cómo evoluciona el estado del entorno tras aplicar una acción.

Equivalencia Funcional Propiedad mediante la cual dos implementaciones del entorno producen el mismo estado resultante ante las mismas entradas.

Golden Logs Trazas generadas desde el entorno Unity utilizadas como referencia para validar la equivalencia del simulador externo.

Política Baseline Política heurística determinista utilizada como referencia experimental.

Política Paramétrica Política cuyo comportamiento depende de un conjunto explícito de pesos ajustables.

Aleatoriedad Acotada Mecanismo que introduce variabilidad controlada mediante un margen relativo ϵ sin alterar el rendimiento competitivo principal.

EvaluationMode Modo experimental donde se fija la semilla del generador aleatorio para garantizar reproducibilidad.

Winrate Proporción de partidas ganadas respecto al total evaluado.

Diferencial de Puntaje Diferencia promedio de puntaje entre dos agentes evaluados.

Latencia de Decisión Tiempo requerido por un agente para seleccionar una acción en un turno.

Reproducibilidad Experimental Capacidad de replicar resultados bajo mismas condiciones iniciales y configuración de semillas.

Imitation Learning Enfoque de aprendizaje supervisado donde un modelo es entrenado para aproximar las decisiones de una política base.

Behavior Cloning Técnica de aprendizaje por imitación donde un modelo (estudiante) se entrena con pares estado-acción generados por otro modelo (profesor o *teacher*).

Destilación de Conocimiento Proceso de transferir el comportamiento aprendido de un modelo complejo a uno más liviano, preservando la mayor fidelidad posible.

Response Table Tabla de mapeo que asocia cada cluster (perfil de oponente) con el especialista más efectivo para ese perfil.

Multi-seed Real Técnica de entrenamiento donde cada genoma o modelo se evalúa contra múltiples semillas simultáneamente, promediando fitness para prevenir sobreajuste a una secuencia determinista particular.

Sistema Adaptativo Combinación de un perfilador (KMeans), una tabla de respuesta y un conjunto de especialistas que permite seleccionar dinámicamente la mejor estrategia en tiempo de ejecución.

Especialista Modelo de IA entrenado específicamente contra un estilo de juego particular (por ejemplo, greedy, denial o spread).

Generalista Modelo de IA que presenta buen rendimiento contra múltiples estilos sin estar especializado en ninguno.

Aprendizaje Online Pleno Forma de adaptación donde se actualizan los pesos o parámetros del modelo durante la partida, utilizando las interacciones en curso como datos de entrenamiento. Requiere ejecutar algoritmos de optimización en tiempo real.

Adaptación Online por Selección Forma de adaptación donde no se actualizan los pesos del modelo en runtime. Se mantiene un catálogo de políticas pre-entrenadas y se selecciona la más adecuada en función del perfil del oponente, requiriendo únicamente inferencia.

Brecha de Datos Relación entre el volumen de datos requerido para entrenar un modelo funcional y el volumen disponible en una sesión real contra un jugador humano. Valores altos indican inviabilidad práctica del entrenamiento online.

Índice General

Lista de Acrónimos	v
Glosario	vi
1. Introducción	1
1.1. Presentación del área y su problemática	1
1.1.1. Contexto de aplicación del proyecto	1
1.1.2. Presentación de los procesos del ámbito del proyecto	1
Evolución del alcance del proyecto	2
1.1.3. Descripción del Problema	3
1.2. Propuesta de solución	4
1.2.1. Separación entre entrenamiento y ejecución	5
1.3. Justificación de la propuesta	5
1.4. Objetivos	6
1.4.1. Objetivo General	6
1.5. Alcance y limitaciones	6
1.5.1. Objetivos Específicos	6
1.5.2. Descripción de actividades	7
1.6. Descripción de los Aspectos Fundamentales de la Metodología Utilizada	8
1.7. Composición del informe	8
2. Bases Teóricas y Antecedentes	10
2.1. Inteligencia Artificial en Videojuegos	10
2.1.1. IA determinista vs. IA adaptativa	11
2.1.2. Aprendizaje online pleno vs. adaptación online por selección	11
2.2. Aprendizaje Automático (<i>Machine Learning</i>)	12
2.2.1. Aprendizaje supervisado	12
2.2.2. Aprendizaje no supervisado	12
2.2.3. Aprendizaje por refuerzo (<i>Reinforcement Learning</i>)	12
2.3. Videojuegos como entornos de evaluación experimental	13
2.4. Ejecución local, modularidad e integración	13
2.4.1. Separación de responsabilidades y arquitectura modular	13

2.4.2.	Entornos equivalentes tipo <i>Gym</i>	14
2.5.	Reproducibilidad experimental e instrumentación	14
2.5.1.	Semillas y control de aleatoriedad	14
2.5.2.	Registro estructurado (<i>logging</i>)	14
2.6.	Costos computacionales y de datos del entrenamiento	15
2.6.1.	Aprendizaje por refuerzo	15
2.6.2.	Neuroevolución	15
2.6.3.	Aprendizaje supervisado (destilación)	15
2.6.4.	Implicaciones para el aprendizaje en tiempo real	15
2.7.	Paradigmas de aprendizaje evaluados en el proyecto	15
2.8.	Métricas de evaluación	16
2.8.1.	Métricas de costo del aprendizaje (nivel primario)	16
2.8.2.	Métricas de desempeño del agente (nivel secundario)	16
3.	Estado del Arte	18
3.1.	IA clásica en videojuegos comerciales	18
3.2.	Adaptación sin aprendizaje online pleno	18
3.2.1.	Caso Alien: Isolation (Creative Assembly, 2014)	18
3.2.2.	Caso Metal Gear Solid V (Konami, 2015)	19
3.2.3.	Síntesis: adaptación por diseño, no por entrenamiento	19
3.3.	Machine Learning en desarrollo y despliegue offline	19
3.3.1.	Caso ARC Raiders (Embark Studios)	19
3.3.2.	Uso industrial de ML en videojuegos	19
3.3.3.	Síntesis de casos industriales	20
3.4.	Brecha entre logros académicos y despliegue real	20
3.5.	Adaptación basada en perfilado de jugador	20
3.6.	Ejecución local versus infraestructura en nube	21
3.7.	Limitaciones identificadas en el estado del arte	21
3.8.	Posicionamiento del presente trabajo	21
4.	Desarrollo del Trabajo	23
4.1.	Arquitectura General del Sistema	23
4.2.	Formalización Matemática del Entorno	24
4.2.1.	Definición del Estado	24
4.2.2.	Función de Puntuación	24
4.2.3.	Regla de Eliminación	25
4.2.4.	Función de Transición	25
4.3.	Diseño del Entorno de Juego	26
4.4.	Middleware y Comunicación	27
4.5.	Representación del Estado y Extracción de Características	28
4.6.	Sistema de Logging e Instrumentación	28
4.7.	Control de Aleatoriedad	28
4.8.	Entorno de Entrenamiento Equivalente (<i>Gym</i>)	29

4.9. Validación del Simulador Externo	29
4.9.1. Pruebas de Equivalencia Funcional	29
4.10. Definición Formal de la Política Baseline	30
4.10.1. Aleatoriedad Acotada	30
4.10.2. Control de Aleatoriedad	30
4.11. Protocolo Experimental	30
4.11.1. Métricas de desempeño	30
4.11.2. Métricas computacionales	31
4.12. Criterios de Viabilidad para Entrenamiento en Tiempo Real	31
4.12.1. Disponibilidad de datos en una sesión real	31
4.12.2. Requisitos de entrenamiento por paradigma	31
4.12.3. Criterio de brecha	31
4.12.4. Criterio de estabilidad	32
5. Resultados	33
5.1. Diseño Experimental	33
5.2. Entrenamiento de Modelos	34
5.3. Análisis de Brecha: Viabilidad del Aprendizaje en Tiempo Real	35
5.3.1. Datos disponibles en una sesión real	36
5.3.2. Interpretación de la brecha	36
5.3.3. Análisis de sensibilidad	37
5.3.4. Conclusión parcial	37
5.4. Calidad de Destilación PPO a DT	37
5.5. Perfilado KMeans	38
5.6. Evaluación de los Tres Sistemas Adaptativos	38
5.6.1. Resultados detallados por sistema	39
5.7. Análisis Estadístico entre Sistemas	39
5.8. Ablation Study: ¿Aporta la Adaptación KMeans?	42
5.8.1. Dominancia en Response Tables	42
5.9. Evaluación Cruzada: Generalización vs Sobreajuste	43
5.10. NEAT: Convergencia, Entrenamiento Extendido y Multi-seed	45
5.11. Discusión	46
5.11.1. Comparativa consolidada de paradigmas	46
5.11.2. Hallazgos principales	48
5.11.3. Implicaciones	49
5.11.4. Limitaciones observadas	49
6. Conclusiones	50
6.1. Conclusiones por Objetivo	50
6.1.1. Medición del costo de aprendizaje	50
6.1.2. Brecha entre entrenamiento offline y sesión real	50
6.1.3. Adaptación por selección como alternativa	51
6.1.4. Desempeño de los sistemas adaptativos	51

6.1.5. Diseño de la plataforma experimental	51
6.2. Conclusión General	51
6.3. Trabajo Futuro	52
6.4. Reflexiones	53
Referencias	54
A. Plan de Trabajo y Carta Gantt	56

Índice de Figuras

1.1. Evolución del alcance del proyecto	3
1.2. Prototipo exploratorio FPS	3
4.1. Arquitectura general del sistema	24
4.2. Prototipo funcional de Knucklebones en Unity	26
4.3. Flujo de decisión por turno	27
4.4. Middleware local en ejecución	27
5.1. Tiempos de entrenamiento por paradigma	35
5.2. Tamaño de modelo por paradigma	35
5.3. Brecha de datos: entrenamiento vs. sesión real	36
5.4. Diferencial de puntaje — DT	40
5.5. Diferencial de puntaje — PPO	40
5.6. Diferencial de puntaje — NEAT	41
5.7. Diferencial de puntaje — Sistemas adaptativos	41
5.8. Mapa de calor de evaluación cruzada	45
5.9. Evolución de fitness NEAT multi-seed	46
5.10. Comparativa consolidada de paradigmas	47
5.11. Winrate por sistema y oponente	48
A.1. Carta Gantt — Parte 1 (abril–julio 2025)	56
A.2. Carta Gantt — Parte 2 (agosto–diciembre 2025)	57
A.3. Carta Gantt — Parte 3 (enero–marzo 2026)	57

Índice de Tablas

3.1. Resumen de enfoques de adaptación en casos industriales y académicos	20
4.1. Ejemplo de transición: el jugador 1 coloca un dado de valor 4 en la columna 1	25
5.1. Tiempos de entrenamiento y tamaño de modelos	34
5.2. Brecha de datos: requerimiento de entrenamiento vs. disponibilidad en sesión real	36
5.3. Análisis de sensibilidad: brecha de datos bajo tres escenarios de sesión humana	37
5.4. Calidad de destilación: accuracy de los DT respecto al PPO teacher	37
5.5. Winrate decidido de cada sistema vs cada oponente (5,000 partidas)	38
5.6. System DT — KMeans + Decision Trees destilados de PPO	39
5.7. System PPO — KMeans + PPO especialistas directos	39
5.8. System NEAT — KMeans + NEAT especialistas directos (multi-seed)	39
5.9. Valores p del z-test entre pares de sistemas (corregidos por Bonferroni)	39
5.10. Ablation study: generalista vs adaptativo vs oráculo	42
5.11. Dominancia de especialistas en las response tables	42
5.12. Evaluación cruzada de NEAT especialistas (winrate decidido)	43
5.13. Evaluación cruzada de PPO especialistas (winrate decidido)	43
5.14. Evaluación cruzada de DT especialistas (winrate decidido)	43
5.15. Resumen de generalización por familia	44
5.16. Efecto de la estrategia de entrenamiento en NEAT vs greedy	45
5.17. Comparativa de paradigmas: rendimiento vs costo	47

Capítulo 1

Introducción

El presente capítulo introduce la problemática abordada en este trabajo, describe la propuesta de solución, justifica su pertinencia y establece los objetivos que guían la investigación. Se presenta el contexto de la inteligencia artificial aplicada a videojuegos, se identifica la pregunta central sobre la viabilidad del aprendizaje en tiempo real bajo restricciones de ejecución local y se plantea la necesidad de un entorno experimental que permita cuantificar dicha viabilidad con evidencia empírica.

1.1. Presentación del área y su problemática

1.1.1. Contexto de aplicación del proyecto

El presente proyecto de titulación no se realiza en convenio con una empresa o institución específica, sino que se enmarca dentro del ámbito de la investigación aplicada en inteligencia artificial y desarrollo de videojuegos. En particular, se enfoca en la evaluación experimental del rendimiento y desempeño del aprendizaje de agentes de IA en un entorno de videojuego ejecutado en hardware local, con el objetivo de cuantificar la viabilidad de distintos paradigmas de aprendizaje bajo restricciones de ejecución offline.

1.1.2. Presentación de los procesos del ámbito del proyecto

En los últimos años, el desarrollo de inteligencias artificiales (IA) aplicadas a videojuegos ha evolucionado desde modelos deterministas —como las máquinas de estados finitos (FSM) o los árboles de decisión— hacia arquitecturas basadas en *Machine Learning* (ML) y *Reinforcement Learning* (RL). Estas tecnologías permiten entrenar agentes con comportamientos más complejos y variados durante la fase de desarrollo, aunque en la mayoría de los casos los modelos se despliegan como políticas fijas en el producto final.

Una pregunta natural que surge de estos avances es: ¿puede una IA aprender en tiempo real del jugador, adaptándose durante la partida sin depender de servidores externos ni de la nube? Esta posibilidad tendría implicaciones significativas para la industria del videojuego, ya que permitiría crear experiencias que se adapten al estilo de cada jugador utilizando exclusivamente los recursos del hardware local. Además, su aplicabilidad se extendería a otros dominios donde se requiera adaptación en tiempo real con recursos limitados, como sistemas de recomendación embebidos o asistentes interactivos.

Sin embargo, los enfoques actuales de entrenamiento de modelos de ML suelen realizarse en entornos de desarrollo separados del producto final, empleando plataformas como Google Colab, Hugging Face Spaces o Unity ML-Agents [1], que frecuentemente requieren conectividad constante y acceso a GPU de alto rendimiento. Esta práctica indica que el entrenamiento —tal como se implementa actualmente— está pensado para la fase de desarrollo, no para ejecutarse dentro del juego durante una sesión con el jugador. No obstante, la viabilidad real de entrenar modelos en tiempo real con los datos que un jugador genera durante una sesión no ha sido cuantificada experimentalmente.

El ciclo convencional de IA en la industria del videojuego presenta las siguientes características:

- **Separación entre entrenamiento y despliegue:** los modelos se entrenan en entornos externos y se integran como políticas fijas en el juego final.
- **Dependencia de volúmenes masivos de datos:** los paradigmas de aprendizaje automático requieren miles a millones de episodios para converger.
- **Ausencia de cuantificación:** no se ha medido experimentalmente cuántos datos genera un jugador humano en una sesión y cómo se compara con lo requerido por los paradigmas de aprendizaje.

El presente proyecto busca abordar esta brecha mediante la creación de un entorno experimental que permita cuantificar el costo real de entrenamiento de distintos paradigmas de IA, contrastar ese costo con los datos disponibles en una sesión de juego real contra un jugador humano, y explorar alternativas de adaptación que sean viables bajo restricciones de ejecución local. Se busca determinar si el aprendizaje en tiempo real es factible, o si existen alternativas prácticas —como la adaptación por selección de especialistas pre-entrenados— que logren el mismo objetivo sin requerir entrenamiento durante la partida.

En este proyecto se opta por un videojuego de reglas simples, turnos discretos y estado completamente observable (*Knucklebones*), lo que permite aislar el comportamiento de la inteligencia artificial, reducir ruido experimental y facilitar mediciones reproducibles.

Evolución del alcance del proyecto

El proyecto fue concebido inicialmente con un enfoque orientado a un videojuego en primera persona (*First Person Shooter*, FPS), dado que este género representa un escenario atractivo para evaluar toma de decisiones y adaptación del comportamiento en agentes controlados por IA. Sin embargo, durante el desarrollo se identificaron dos factores determinantes para reformular el alcance:

- **Complejidad de implementación y riesgo técnico:** un FPS introduce múltiples variables simultáneas (física, navegación, percepción parcial, sincronización en tiempo real, animaciones, balance de combate y diseño de niveles), lo que incrementa el riesgo de que los resultados experimentales se vean afectados por aspectos ajenos al objetivo principal de la investigación.
- **Irrelevancia del género para el objetivo experimental:** el objetivo de este trabajo no es evaluar la calidad audiovisual o el diseño de un género específico, sino estudiar el desempeño, costo computacional y reproducibilidad de distintos enfoques de IA en un entorno de videojuego. Por lo tanto, la naturaleza del entorno debe privilegiar control experimental y medición.

En consecuencia, se optó por un entorno de reglas simples, turnos discretos y estado completamente observable (*Knucklebones*). Esta elección permite aislar el problema de decisión, reducir ruido experimental y fortalecer la reproducibilidad de resultados, manteniendo el foco del proyecto en la evaluación

del costo de aprendizaje y mecanismos de adaptación bajo restricciones de ejecución local. La Figura 1.1 ilustra esta transición.

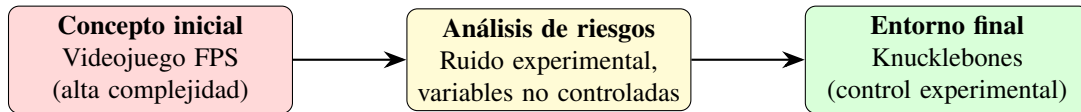


Figura 1.1: Evolución del alcance del proyecto: del concepto FPS al entorno controlado Knucklebones.

La Figura 1.2 muestra el prototipo FPS original, cuya complejidad motivó la transición hacia un entorno de menor ruido experimental.



Figura 1.2: Prototipo exploratorio inicial del entorno FPS en Unity, desarrollado durante la primera fase del proyecto. Esta versión se utilizó para validar jugabilidad base y la viabilidad de integrar métricas en hardware local, pero fue descartada como entorno experimental principal por su alta complejidad y bajo control metodológico.

1.1.3. Descripción del Problema

Actualmente, existe una percepción extendida —tanto en la industria del videojuego como en la comunidad académica— de que las inteligencias artificiales en videojuegos pueden o deberían “aprender en tiempo real” mientras interactúan con el jugador. Casos recientes como ARC Raiders han sido promocionados sugiriendo que los enemigos exhiben comportamientos emergentes aprendidos durante el gameplay, cuando la evidencia pública disponible indica que los modelos fueron entrenados durante el desarrollo y desplegados como políticas fijas.

Esta percepción genera una brecha entre la expectativa y la realidad técnica. Los paradigmas de aprendizaje automático más utilizados —aprendizaje por refuerzo, neuroevolución, aprendizaje supervisado— requieren volúmenes de datos y ciclos de entrenamiento que podrían exceder lo disponible en una sesión

de juego contra un jugador humano. Sin embargo, esta brecha no ha sido cuantificada de forma sistemática bajo restricciones de hardware local.

Por otra parte, si bien existen herramientas y entornos para entrenar agentes de IA en videojuegos (como OpenAI Gym [2], Unity ML-Agents [1] o Stable-Baselines3 [3]), estas están orientadas principalmente al proceso de entrenamiento offline, no a cuantificar la brecha entre los datos requeridos para entrenar y los datos que un jugador humano genera en una sesión real. Esta cuantificación resulta necesaria para determinar la viabilidad del aprendizaje en tiempo real.

El problema central que aborda este proyecto radica en la ausencia de evidencia experimental que cuantifique la viabilidad del aprendizaje de IA en tiempo real bajo restricciones de ejecución local, y en la necesidad de explorar alternativas —como la adaptación por selección de especialistas pre-entrenados— que ofrezcan adaptabilidad sin depender de entrenamiento online continuo.

1.2. Propuesta de solución

La propuesta de solución consiste en diseñar e implementar un entorno experimental que permita determinar, con evidencia cuantitativa, si es factible que una inteligencia artificial aprenda en tiempo real del jugador en un videojuego ejecutado en hardware local. Para ello, se construye una plataforma basada en un juego de turnos discretos (*Knucklebones*), donde se entrenan tres paradigmas representativos de IA y se mide rigurosamente el costo de dicho entrenamiento, contrastándolo con los datos que un jugador humano genera en una sesión real.

El proyecto plantea dos líneas complementarias:

- (a) **Cuantificación de la viabilidad del aprendizaje en tiempo real:** entrenar tres paradigmas de IA (aprendizaje por refuerzo, neuroevolución, aprendizaje supervisado destilado) de forma offline, midiendo los datos requeridos, el tiempo de entrenamiento y los recursos computacionales consumidos. Estos datos permiten evaluar la brecha entre los requisitos de cada paradigma y lo disponible en una sesión de juego real contra un jugador humano, determinando si el aprendizaje en tiempo real es viable o no.
- (b) **Adaptación por selección como alternativa:** dado que la hipótesis inicial sugiere que el aprendizaje en tiempo real podría no ser viable con los paradigmas convencionales, se diseña e implementa un mecanismo de adaptación que no requiera reentrenamiento en tiempo de ejecución, basado en perfilado del oponente (clustering no supervisado) y selección de especialistas pre-entrenados. Este mecanismo se evalúa como alternativa práctica al entrenamiento online continuo.

Para ello, se emplea el motor de desarrollo Unity como entorno de ejecución de reglas, junto con una arquitectura modular diseñada desde el inicio con separación estricta entre el motor del juego (responsable de reglas y transición de estado) y los agentes de IA (responsables exclusivamente de la selección de acciones). Un middleware HTTP/JSON permite la comunicación entre ambas capas. Esta separación modular fue una decisión de diseño planificada, no una limitación descubierta durante el desarrollo.

El sistema incluye instrumentación capaz de registrar métricas tales como latencia de decisión, carga computacional, tiempo de entrenamiento y tamaño de modelo. Estas métricas permiten evaluar de manera objetiva tanto el desempeño de cada enfoque como su costo, determinando su viabilidad bajo restricciones de hardware local.

La ejecución local en hardware convencional garantiza accesibilidad y reduce dependencia de recursos remotos. El diseño modular permite incorporar nuevos agentes o configuraciones experimentales con bajo costo adicional.

1.2.1. Separación entre entrenamiento y ejecución

Desde el diseño inicial del proyecto se estableció una arquitectura modular que separa la lógica del videojuego de los agentes de IA. El videojuego se mantiene como entorno estable que expone estado y recibe acciones, mientras que el entrenamiento se realiza fuera del motor utilizando un entorno equivalente tipo *Gym* [2]. Esta decisión de diseño responde a criterios de reproducibilidad, trazabilidad y eficiencia experimental.

El entorno equivalente permite ejecutar partidas a alta velocidad, generar datasets y entrenar modelos con mayor eficiencia, manteniendo consistencia con las reglas del juego y conservando la ejecución en hardware local. Esta separación no contradice el objetivo de evaluar la viabilidad del aprendizaje en tiempo real; por el contrario, permite medirla de forma rigurosa: si el entrenamiento offline —en condiciones óptimas de velocidad y volumen de datos— ya requiere órdenes de magnitud más datos de los disponibles en una sesión real, el entrenamiento en tiempo real durante la partida resulta inviable.

De esta manera, el sistema mantiene equivalencia funcional entre el entorno de entrenamiento y el entorno de evaluación, garantizando la validez experimental de los resultados obtenidos.

1.3. Justificación de la propuesta

El desarrollo de este proyecto se justifica por la necesidad de responder, con evidencia experimental, a una pregunta que la industria del videojuego y la comunidad académica asumen implícitamente pero no han cuantificado: ¿es viable que una inteligencia artificial aprenda en tiempo real dentro de un videojuego ejecutado en hardware local?

Desde un punto de vista técnico, la propuesta aborda esta pregunta mediante la medición rigurosa del costo de entrenamiento de tres paradigmas representativos (aprendizaje por refuerzo, neuroevolución, aprendizaje supervisado), cuantificando los datos, el tiempo y los recursos requeridos, y contrastándolos con lo disponible en una sesión real. La separación modular entre juego y agentes permite aislar el rendimiento del aprendizaje de factores externos, y el diseño del sistema habilita la evaluación de alternativas como la adaptación por selección.

En términos de reproducibilidad, la evaluación y el entrenamiento se realizan localmente en hardware doméstico, sin requerir infraestructura externa. Esto garantiza que los resultados puedan ser replicados por otros investigadores con recursos similares y elimina dependencias de servicios en la nube.

Funcionalmente, el sistema incluye instrumentación integrada que registra no solo el desempeño del agente durante el juego (winrate, latencia), sino también el costo del proceso de aprendizaje (tiempo de entrenamiento, datos requeridos, tamaño de modelo). Esta doble medición permite un análisis de viabilidad fundamentado en evidencia.

Desde una perspectiva de investigación, este proyecto contribuye con evidencia cuantitativa sobre una pregunta que, si bien es relevante tanto para la industria del videojuego como para cualquier dominio

que requiera adaptación en tiempo real con recursos limitados, ha sido poco abordada de forma experimental: ¿cuánto cuesta realmente que una IA aprenda, y es posible hacerlo con los datos que genera un usuario en una sesión real?

1.4. Objetivos

1.4.1. Objetivo General

Evaluar el rendimiento y desempeño del aprendizaje de distintas inteligencias artificiales en un entorno de videojuego ejecutado completamente offline, cuantificando la brecha entre los requisitos de entrenamiento de tres paradigmas representativos y los datos disponibles en una sesión real contra un jugador humano, con el fin de determinar la viabilidad del aprendizaje en tiempo real y evaluar la adaptación por selección de especialistas pre-entrenados como alternativa práctica, sin depender de infraestructura en la nube.

1.5. Alcance y limitaciones

El trabajo abarca los siguientes elementos:

- Diseño e implementación de una arquitectura modular que separa el videojuego de los agentes de IA.
- Entrenamiento de especialistas en tres paradigmas: PPO (aprendizaje por refuerzo), NEAT (neuro-evolución) y DT (destilación por *Behavior Cloning*).
- Medición del costo de entrenamiento de cada paradigma (datos, tiempo, recursos).
- Análisis de la brecha entre los datos requeridos y los disponibles en una sesión contra un jugador humano.
- Implementación de un mecanismo de adaptación por selección (KMeans + response table + especialistas).
- Integración del sistema completo en Unity mediante middleware HTTP/JSON.

Como limitaciones deliberadas:

- No se implementa entrenamiento online completo dentro del motor del juego. Los resultados documentan cuantitativamente por qué dicho entrenamiento no es viable con los paradigmas evaluados.
- La estimación de datos disponibles en una sesión humana (10–100 partidas) es un supuesto operativo derivado de observación práctica, no un estudio formal de comportamiento de usuario.
- Los oponentes de entrenamiento y evaluación son heurísticos diseñados, no jugadores humanos reales.
- El entorno experimental es un juego de reglas simples (Knucklebones); la generalización a dominios más complejos queda como trabajo futuro.

1.5.1. Objetivos Específicos

- Diseñar una arquitectura modular que permita incorporar y evaluar diferentes enfoques de inteligencia artificial sin modificar las reglas del videojuego.

- Implementar un prototipo funcional basado en un videojuego de turnos discretos que opere como plataforma de prueba para la ejecución y evaluación reproducible de agentes.
- Entrenar y evaluar tres paradigmas de aprendizaje —aprendizaje por refuerzo (PPO), neuroevolución (NEAT) y aprendizaje supervisado destilado (DT)— midiendo el costo de entrenamiento en términos de datos requeridos, tiempo y recursos computacionales.
- Cuantificar la brecha entre los requisitos de entrenamiento de cada paradigma y los datos disponibles en una sesión real contra un jugador humano, para determinar la viabilidad del aprendizaje en tiempo real.
- Implementar y evaluar un mecanismo de adaptación por selección de especialistas pre-entrenados como alternativa al entrenamiento online continuo.
- Integrar los agentes en el videojuego Unity mediante un middleware desacoplado, verificando la viabilidad operativa del sistema completo.

1.5.2. Descripción de actividades

- **Diseñar una arquitectura modular para agentes intercambiables.**
 - Diseño de la estructura modular del sistema y definición de interfaz estado/acción.
 - Elaboración de esquemas lógicos y diagramas de componentes.
 - Definición de flujos para integración de agentes y centralización de extracción de características.
- **Desarrollar el prototipo de videojuego como entorno de evaluación.**
 - Construcción del entorno base del juego seleccionado.
 - Implementación del protocolo de observación por turno y aplicación de acciones.
 - Aseguramiento de condiciones de control experimental (semillas, registro estructurado).
- **Entrenar y medir el costo de aprendizaje de cada paradigma.**
 - Entrenamiento de especialistas PPO mediante aprendizaje por refuerzo con entorno acelerado.
 - Entrenamiento de especialistas NEAT mediante neuroevolución con evaluación multi-seed.
 - Destilación de PPO a árboles de decisión mediante *Behavior Cloning*.
 - Registro de datos requeridos, tiempo de entrenamiento y recursos computacionales por cada paradigma.
- **Cuantificar la brecha de viabilidad para entrenamiento en tiempo real.**
 - Estimación de los datos disponibles en una sesión real contra un jugador humano.
 - Cálculo de la relación entre datos requeridos y datos disponibles por paradigma.
 - Análisis de convergencia con datos limitados (evidencia de entrenamiento insuficiente).
- **Implementar y evaluar el mecanismo de adaptación por selección.**
 - Entrenamiento de perfilador KMeans con datos de oponentes heurísticos.
 - Construcción de tablas de respuesta por evaluación de especialistas por cluster.
 - Evaluación comparativa del sistema adaptativo frente a generalistas fijos (*ablation study*).
- **Integrar el sistema completo en el videojuego Unity.**
 - Implementación de middleware HTTP/JSON para comunicación Unity–Python.
 - Verificación de equivalencia funcional entre entorno de simulación y motor de juego.
 - Medición de latencia operativa del sistema integrado.

1.6. Descripción de los Aspectos Fundamentales de la Metodología Utilizada

La presente investigación se enmarcó dentro del paradigma de la investigación aplicada, combinando elementos de análisis experimental, diseño iterativo de software y evaluación comparativa. Debido a la naturaleza técnica del proyecto, que implicó implementar un entorno de evaluación reproducible, instrumentado y controlado para medir de forma cuantificable el comportamiento y el costo computacional de agentes de IA bajo condiciones homogéneas, se adoptó una metodología de carácter evolutivo y orientada a prototipos.

El enfoque metodológico se fundamentó en tres pilares principales:

- **Investigación exploratoria y revisión técnica:** permitió comprender el estado actual de las tecnologías involucradas, incluyendo IA para videojuegos, modularidad de software, instrumentación y limitaciones de hardware local.
- **Desarrollo incremental de prototipos:** el videojuego y los módulos del sistema fueron construidos de manera progresiva, incorporando funcionalidades esenciales en ciclos iterativos. Este enfoque facilitó la validación temprana de componentes, redujo riesgos asociados a la complejidad del sistema y permitió evaluar continuamente la factibilidad técnica del proyecto.
- **Experimentación y análisis de rendimiento:** los enfoques de IA fueron evaluados mediante métricas cuantitativas, incluyendo latencia de decisión, uso de CPU/memoria, estabilidad y métricas de desempeño dentro del juego. Esta fase permitió comparar enfoques bajo condiciones homogéneas y completamente offline.

La elección de una metodología evolutiva respondió a la incertidumbre inherente al desarrollo de sistemas de IA y al nivel de riesgo técnico asociado a integración, medición y control experimental. Este enfoque permitió ajustar el diseño del sistema de forma continua y corregir desviaciones durante el avance de la investigación.

En la primera etapa se desarrolló un levantamiento del marco tecnológico y conceptual. Luego se diseñó e implementó una arquitectura modular que soporta la integración de agentes y registro de métricas. Posteriormente, se construyó el prototipo funcional como plataforma de pruebas, sobre el cual se integraron y evaluaron distintos enfoques de IA. Finalmente, se realizó un análisis comparativo de resultados con el fin de determinar la factibilidad técnica y operativa de los enfoques evaluados en un entorno de videojuego ejecutado completamente en hardware local.

De esta manera, la metodología seleccionada garantizó una progresión controlada del proyecto, favoreció la validación temprana de hipótesis y aseguró que los objetivos planteados pudieran ser alcanzados mediante un proceso sistemático, reproducible y fundamentado en evidencia experimental.

1.7. Composición del informe

El presente trabajo se encuentra dividido en seis capítulos. A continuación se describe brevemente el contenido de cada uno de ellos.

- (a) **Introducción:** presentación del problema, objetivos, alcance y metodología general del trabajo.

- (b) **Bases Teóricas:** fundamentos sobre IA en videojuegos, aprendizaje automático, distinción entre aprendizaje online y adaptación por selección, costos de entrenamiento y reproducibilidad experimental.
- (c) **Estado del Arte:** revisión de enfoques existentes en la industria y la academia, con énfasis en la viabilidad del aprendizaje en tiempo real y las prácticas de adaptación utilizadas.
- (d) **Desarrollo del trabajo:** diseño e implementación de la plataforma, arquitectura modular, instrumentación, flujo experimental y criterios de viabilidad para entrenamiento en tiempo real.
- (e) **Resultados:** medición de costos de aprendizaje, análisis de brecha de datos, evaluación de sistemas adaptativos, discusión de viabilidad y comparación de paradigmas.
- (f) **Conclusiones:** síntesis de hallazgos sobre viabilidad del aprendizaje, validación de la adaptación por selección y proyecciones.

Además, al final del informe se adjuntan las referencias bibliográficas utilizadas en el proceso de investigación.

Capítulo 2

Bases Teóricas y Antecedentes

El presente capítulo tiene por finalidad describir los fundamentos conceptuales, tecnológicos y metodológicos asociados al uso de inteligencia artificial aplicada a videojuegos, con énfasis en la evaluación del rendimiento y desempeño del aprendizaje bajo restricciones de ejecución local. Se presentan conceptos clave sobre IA en videojuegos, aprendizaje automático, la distinción entre aprendizaje online y adaptación por selección, los costos asociados al entrenamiento de modelos, y nociones de modularidad y reproducibilidad experimental.

A través de esta revisión se establece un marco conceptual que permite comprender: (i) por qué los videojuegos constituyen entornos relevantes para evaluar el aprendizaje de agentes, (ii) qué técnicas de IA resultan factibles bajo recursos computacionales limitados, (iii) cuáles son los costos reales del proceso de aprendizaje en cada paradigma, y (iv) qué alternativas existen cuando el entrenamiento en tiempo real no es viable.

2.1. Inteligencia Artificial en Videojuegos

La inteligencia artificial (IA) ha sido un componente central en el desarrollo de videojuegos desde sus primeras generaciones [4]. Su propósito principal es controlar el comportamiento de personajes no jugadores (*Non-Player Characters*, NPC), administrar dinámicas del entorno y ofrecer experiencias desafiantes y coherentes para el jugador humano.

A diferencia de otras áreas de la IA, en videojuegos la toma de decisiones suele estar condicionada por restricciones de tiempo real, estabilidad del sistema y criterios de diseño (*game design*); por lo tanto, la IA debe ser no solo efectiva, sino también controlable, predecible en su estructura y suficientemente interpretable para evitar comportamientos indeseados o desbalanceados.

Tradicionalmente, la IA en videojuegos se ha implementado mediante enfoques deterministas y basados en reglas, entre los cuales destacan:

- Máquinas de estados finitos (FSM).
- Árboles de decisión.
- Árboles de comportamiento (*Behavior Trees*).
- Sistemas basados en reglas y heurísticas.

Estos modelos presentan ventajas relevantes: bajo costo computacional, facilidad de depuración y control explícito del comportamiento. Sin embargo, su principal limitación es la ausencia de adaptación genuina: el agente no modifica su política a partir de la experiencia, y su comportamiento tiende a volverse predecible ante jugadores recurrentes, lo que puede afectar la rejugabilidad.

2.1.1. IA determinista vs. IA adaptativa

En términos generales, puede distinguirse entre:

- **IA determinista:** ejecuta políticas fijas predefinidas (por ejemplo, reglas o árboles con parámetros constantes), sin mecanismos de actualización basados en datos.
- **IA adaptativa:** modifica su comportamiento en función del jugador o del historial de interacción. Esta adaptación puede lograrse mediante reglas dinámicas diseñadas por el desarrollador o mediante técnicas de aprendizaje automático.

Una consideración relevante para esta investigación es que el término “adaptativo” no implica necesariamente entrenamiento profundo en tiempo real. En videojuegos, un enfoque adaptativo puede consistir en selección de estrategias, ajuste de parámetros o cambio de política en función de patrones observables, manteniendo control de diseño y costo computacional acotado.

En el contexto de este trabajo, la adaptatividad se aborda desde una perspectiva centrada en el costo del aprendizaje: se investiga cuánto cuesta entrenar cada paradigma y se evalúa la adaptación por selección como alternativa al entrenamiento en tiempo real.

2.1.2. Aprendizaje online pleno vs. adaptación online por selección

Una distinción central para esta investigación es la diferencia entre dos formas de lograr comportamiento adaptativo en tiempo de ejecución:

- **Aprendizaje online pleno:** consiste en actualizar los parámetros o pesos del modelo *durante la partida*, utilizando las interacciones en curso como datos de entrenamiento. Esto requiere ejecutar algoritmos de optimización (retropropagación, evaluación de fitness, etc.) en tiempo real, con los costos computacionales y de datos asociados. Ejemplos: fine-tuning de una red neuronal con los turnos jugados, evolución de genomas con las partidas observadas.
- **Adaptación online por selección:** no se actualizan los pesos del modelo en runtime. En su lugar, se mantiene un catálogo de políticas pre-entrenadas y se selecciona la más adecuada en función de un perfil del oponente construido en tiempo real. Esto requiere únicamente inferencia (microsegundos) y un módulo de perfilado ligero (por ejemplo, clustering), sin costo de entrenamiento durante la partida.

Esta distinción es relevante porque el término “IA adaptativa” en videojuegos frecuentemente se utiliza sin precisar cuál de estos dos mecanismos se está describiendo. El presente trabajo evalúa experimentalmente la viabilidad del primero y propone el segundo como alternativa demostrada.

2.2. Aprendizaje Automático (*Machine Learning*)

El *Machine Learning* (ML) es una rama de la IA basada en la idea de que un sistema puede aprender patrones a partir de datos, mejorar su rendimiento con la experiencia y generalizar hacia nuevas situaciones sin ser programado explícitamente con reglas exhaustivas. En el contexto de videojuegos, el ML puede utilizarse para:

- predecir acciones o preferencias del jugador,
- clasificar estilos de juego (por ejemplo, agresivo o defensivo),
- aprender políticas de decisión mediante interacción con el entorno,
- aproximar funciones de evaluación o comportamiento.

Las principales categorías de ML se describen a continuación.

2.2.1. Aprendizaje supervisado

El aprendizaje supervisado entrena un modelo utilizando ejemplos etiquetados x, y , donde x representa un conjunto de características (*features*) y y una salida objetivo (clase o valor). En videojuegos, esto resulta útil cuando se dispone de datos registrados (por ejemplo, decisiones por turno) y se busca:

- predecir una acción a partir del estado,
- imitar una política base (aprendizaje por imitación),
- clasificar eventos o patrones del juego.

Una ventaja clave del aprendizaje supervisado es su eficiencia tanto en entrenamiento como en inferencia (dependiendo del modelo), además de su potencial interpretabilidad en modelos como los árboles de decisión.

2.2.2. Aprendizaje no supervisado

El aprendizaje no supervisado busca identificar estructura en datos sin etiquetas explícitas. Su aplicación en videojuegos incluye:

- agrupamiento (*clustering*) de jugadores según patrones de comportamiento,
- reducción de dimensionalidad,
- detección de anomalías o estilos de juego.

En el contexto de IA adaptativa, un modelo no supervisado puede actuar como módulo auxiliar que detecta perfiles y gatilla cambios de estrategia, sin necesariamente ejecutar directamente la política de juego.

2.2.3. Aprendizaje por refuerzo (*Reinforcement Learning*)

El aprendizaje por refuerzo (RL) modela el problema como un agente que interactúa con un entorno y recibe recompensas, aprendiendo una política $\pi a | s$ que maximiza el retorno esperado [5]. Sus elementos básicos son:

- **Agente:** entidad que selecciona acciones.
- **Entorno:** sistema con el que el agente interactúa.

- **Estado s :** representación de la situación actual.
- **Acción a :** decisión aplicada al entorno.
- **Recompensa r :** retroalimentación numérica que guía el aprendizaje.
- **Política π :** regla de decisión del agente.

RL es especialmente relevante en videojuegos por su naturaleza secuencial y por la posibilidad de generar episodios de interacción. No obstante, en escenarios de ejecución local, el entrenamiento de RL puede resultar costoso en tiempo y recursos computacionales, por lo que suele requerir simulaciones aceleradas o entrenamiento fuera del motor del juego para mantener un ciclo de iteración razonable.

2.3. Videojuegos como entornos de evaluación experimental

Los videojuegos pueden funcionar como entornos experimentales adecuados para evaluar agentes por diversas razones:

- **Entornos controlados:** reglas explícitas y estados formalizables.
- **Interacciones repetibles:** posibilidad de re-ejecutar episodios con semillas controladas.
- **Retroalimentación inmediata:** métricas observables por turno o por episodio.
- **Simulación acelerada:** en juegos por turnos, es posible ejecutar múltiples episodios rápidamente fuera del motor.

En esta investigación se prioriza un entorno con reglas simples y estado completamente observable, de modo que la comparación entre agentes sea atribuible al enfoque de IA y no a factores externos como percepción parcial compleja, navegación continua o variabilidad no controlada.

2.4. Ejecución local, modularidad e integración

La ejecución local impone restricciones prácticas relacionadas con capacidad de CPU/GPU, memoria disponible, estabilidad del proceso y latencias tolerables. En este escenario, el diseño arquitectónico se vuelve crítico para asegurar viabilidad y comparabilidad.

2.4.1. Separación de responsabilidades y arquitectura modular

Un principio de ingeniería relevante es la separación de responsabilidades (*separation of concerns*): el juego debe encargarse de ejecutar reglas y transiciones de estado, mientras que los agentes deben encargarse exclusivamente de seleccionar acciones. Esta separación habilita una arquitectura *plug-and-play*, donde distintos agentes pueden intercambiarse sin modificar el núcleo del videojuego.

En esta investigación, la modularidad se considera un requisito metodológico, dado que permite:

- comparar agentes bajo reglas idénticas,
- centralizar el cálculo de características para evitar inconsistencias,
- registrar métricas de forma uniforme,
- reducir acoplamiento y complejidad de mantenimiento.

Este principio se operacionaliza mediante el concepto de *juego inmutable*, donde el motor se limita a ejecutar reglas y transiciones de estado, mientras que cualquier lógica específica de IA permanece desacoplada y reemplazable.

2.4.2. Entornos equivalentes tipo *Gym*

Debido a restricciones de tiempo y costo de iteración, resulta práctico entrenar o evaluar ciertos modelos fuera del motor del juego mediante un entorno equivalente tipo *Gym* [2]. Este entorno replica las reglas relevantes y expone una interfaz estándar (observación, acción, recompensa y terminación), permitiendo ejecutar episodios a mayor velocidad y generar datasets de forma eficiente.

Este enfoque es coherente con la ejecución local, ya que tanto el entrenamiento como la evaluación continúan realizándose en el hardware del usuario, pero desacoplados del motor, lo cual facilita la instrumentación y la reproducibilidad.

Para preservar la validez experimental, el entorno equivalente debe cumplir equivalencia funcional respecto al motor del juego. En términos prácticos, esto implica que, para una misma transición estado–acción, ambos entornos produzcan el mismo estado resultante. Este criterio puede verificarse mediante trazas de referencia (*golden logs*) y pruebas sistemáticas de transición.

2.5. Reproducibilidad experimental e instrumentación

En trabajos experimentales, la reproducibilidad es fundamental para validar resultados y comparar enfoques. En el contexto de videojuegos, esto requiere controlar fuentes de aleatoriedad y asegurar trazabilidad completa de ejecuciones.

2.5.1. Semillas y control de aleatoriedad

El uso de semillas permite controlar generadores de números aleatorios (RNG) tanto en el entorno como en los agentes. Al fijar una semilla, una secuencia de eventos aleatorios (por ejemplo, valores de dados o desempates) puede repetirse, habilitando comparaciones justas entre agentes bajo condiciones homogéneas.

2.5.2. Registro estructurado (*logging*)

El registro estructurado de decisiones por turno permite:

- reconstruir partidas completas,
- analizar causas de decisiones,
- medir latencia y costo computacional,
- construir datasets para entrenamiento supervisado,
- comparar estabilidad y variabilidad entre ejecuciones.

En este trabajo, el registro estructurado se plantea como componente metodológico central, ya que actúa como evidencia experimental y soporte para el análisis cuantitativo.

2.6. Costos computacionales y de datos del entrenamiento

El proceso de entrenamiento de modelos de aprendizaje automático implica costos que varían significativamente según el paradigma utilizado. Comprender estos costos es fundamental para evaluar la viabilidad de cada enfoque bajo restricciones de ejecución local.

2.6.1. Aprendizaje por refuerzo

El entrenamiento de agentes por RL requiere que el agente interactúe con el entorno durante cientos de miles o millones de pasos (*timesteps*). Algoritmos como PPO [6] utilizan rollouts (secuencias de interacción) para estimar gradientes de política. El volumen de datos necesario depende de la complejidad del entorno, pero en la práctica se requieren entre 10^5 y 10^7 timesteps para juegos simples [5]. Este volumen es generado por el propio proceso de entrenamiento interactuando con un simulador, no por jugadores humanos.

2.6.2. Neuroevolución

Los algoritmos evolutivos como NEAT [7] evalúan poblaciones completas de candidatos (genomas) durante múltiples generaciones. El costo total se expresa como: generaciones $\times$ tamaño de población $\times$ episodios por genoma. Con poblaciones de 150 individuos, 300 generaciones y 50–100 episodios por evaluación, el volumen total de episodios puede alcanzar millones. A diferencia de RL, la evaluación de cada individuo es independiente y paralelizable, pero el volumen total sigue siendo elevado.

2.6.3. Aprendizaje supervisado (destilación)

El aprendizaje supervisado por imitación (*Behavior Cloning*) [8] es computacionalmente ligero en la fase de entrenamiento del modelo (segundos), pero depende de un dataset generado previamente. Este dataset requiere un teacher ya entrenado (por ejemplo, un agente PPO), lo que traslada el costo a una etapa anterior. El volumen de datos etiquetados necesario es del orden de 10^5 – 10^6 muestras para lograr fidelidad superior al 90 %.

2.6.4. Implicaciones para el aprendizaje en tiempo real

Los costos descritos permiten anticipar que el entrenamiento en tiempo real —donde los datos provienen exclusivamente de la interacción con un jugador humano durante una sesión de juego— enfrenta una limitación fundamental: el jugador humano genera órdenes de magnitud menos datos de los requeridos por cualquiera de estos paradigmas. Esta brecha se cuantifica experimentalmente en el Capítulo 5.

2.7. Paradigmas de aprendizaje evaluados en el proyecto

Con el objetivo de evaluar el rendimiento y desempeño del aprendizaje bajo restricciones locales, se seleccionaron tres paradigmas representativos que cubren distintas familias de aprendizaje automático y

presentan costos de entrenamiento contrastados:

- **Aprendizaje por refuerzo (PPO):** el agente aprende una política mediante interacción con el entorno, optimizando el retorno esperado. Representa el paradigma de mayor costo de datos.
- **Neuroevolución (NEAT):** el agente evoluciona mediante algoritmos genéticos que optimizan la topología y pesos de redes neuronales. Representa un costo intermedio con alta paralelizabilidad.
- **Aprendizaje supervisado destilado (DT):** el agente aprende por imitación de un teacher previamente entrenado (PPO), produciendo un modelo ligero e interpretable. Representa el menor costo de entrenamiento directo, pero depende de un teacher.

Adicionalmente, el proyecto emplea componentes auxiliares que no constituyen paradigmas de aprendizaje independientes:

- **Heurísticas:** políticas deterministas diseñadas manualmente que sirven como baseline y como oponentes de entrenamiento.
- **Perfilador KMeans:** módulo no supervisado que clasifica el estilo del oponente para habilitar la selección de especialistas.

Esta selección permite estudiar la relación entre desempeño y costo computacional, y cuantificar qué se obtiene realmente al incorporar aprendizaje respecto a una política base. La combinación de los tres paradigmas con el perfilador KMeans conforma los sistemas adaptativos evaluados.

2.8. Métricas de evaluación

Para medir el rendimiento y desempeño del aprendizaje se requieren métricas en dos niveles complementarios: el costo del proceso de aprendizaje y el desempeño del agente resultante.

2.8.1. Métricas de costo del aprendizaje (nivel primario)

- **Datos requeridos:** volumen de muestras, episodios o timesteps necesarios para que el modelo converja a una política funcional.
- **Tiempo de entrenamiento:** duración del proceso de entrenamiento medida en segundos de reloj (*wall time*).
- **Uso de CPU y memoria:** costo de recursos computacionales durante el entrenamiento.
- **Tamaño del modelo resultante:** espacio en disco del artefacto entrenado.
- **Brecha de datos:** relación entre los datos requeridos y los disponibles en una sesión real contra un jugador humano.

2.8.2. Métricas de desempeño del agente (nivel secundario)

- **Tasa de victoria (*winrate*):** proporción de partidas ganadas, que confirma que el agente entrenado es funcional.
- **Diferencial de puntaje:** diferencia promedio entre el puntaje del agente y el del oponente.
- **Estabilidad:** variabilidad de resultados entre ejecuciones repetidas.
- **Latencia de decisión:** tiempo requerido para seleccionar una acción por turno (relevante para la viabilidad operativa).

El análisis conjunto de ambos niveles permite estudiar el compromiso (*trade-off*) entre el costo del aprendizaje y la calidad del agente resultante, determinando si la inversión en entrenamiento se justifica y bajo qué condiciones es viable.

Capítulo 3

Estado del Arte

El presente capítulo revisa los enfoques actuales utilizados para implementar inteligencia artificial en videojuegos, tanto en contextos académicos como industriales. Se analizan técnicas deterministas, enfoques basados en aprendizaje automático y mecanismos de adaptación al jugador, con especial atención a la viabilidad del aprendizaje en tiempo real y a las alternativas implementadas en la práctica bajo restricciones de hardware local.

3.1. IA clásica en videojuegos comerciales

La mayoría de los videojuegos comerciales continúan utilizando modelos deterministas como máquinas de estados finitos (FSM), árboles de comportamiento y sistemas basados en reglas [4]. Estos enfoques permiten un control preciso del comportamiento, bajo consumo computacional y alta estabilidad en tiempo real.

Ejemplos representativos incluyen la saga *Halo* (Bungie/343 Industries), donde los enemigos utilizan árboles de comportamiento para coordinar tácticas de grupo; *F.E.A.R.* (Monolith, 2005), reconocido por su sistema de planificación GOAP (*Goal-Oriented Action Planning*); y *The Last of Us* (Naughty Dog, 2013), donde la IA de los compañeros emplea FSM con ajustes contextuales. En todos estos casos, la IA prioriza coherencia narrativa, controlabilidad y estabilidad sobre el aprendizaje autónomo. Su bajo costo computacional y facilidad de depuración los mantienen vigentes en producción comercial.

3.2. Adaptación sin aprendizaje online pleno

En varios títulos comerciales se implementan mecanismos de adaptación al jugador que, a pesar de generar la percepción de aprendizaje, no actualizan pesos de modelos durante la ejecución. A continuación se presentan tres casos representativos.

3.2.1. Caso Alien: Isolation (Creative Assembly, 2014)

El Xenomorfo de *Alien: Isolation* no entrena un modelo de aprendizaje automático. En su lugar, detecta patrones del jugador y desbloquea contramedidas predefinidas. Si el jugador se esconde frecuentemente

debajo de mesas, el Alien incrementa la probabilidad de buscar en esas ubicaciones. Este mecanismo constituye adaptación por reglas condicionales, no por aprendizaje.

3.2.2. Caso Metal Gear Solid V (Konami, 2015)

En MGSV, los soldados enemigos reaccionan a tendencias del jugador: muchos disparos a la cabeza provocan que los soldados usen cascos; jugar frecuentemente de noche resulta en que los enemigos adquieran visores nocturnos. Este sistema representa un enfoque adaptativo basado en contadores y umbrales, sin componentes de aprendizaje automático.

3.2.3. Síntesis: adaptación por diseño, no por entrenamiento

Estos casos evidencian que la adaptación al jugador en videojuegos comerciales se implementa predominantemente mediante reglas condicionales, contadores y respuestas predefinidas por los diseñadores. No se actualizan pesos ni se ejecutan algoritmos de optimización en runtime. La adaptación se logra por diseño, no por aprendizaje.

3.3. Machine Learning en desarrollo y despliegue offline

3.3.1. Caso ARC Raiders (Embark Studios)

ARC Raiders, desarrollado por Embark Studios [9], fue analizado como caso industrial contemporáneo asociado discursivamente a inteligencia artificial “emergente”. La evidencia pública revisada sugiere que ARC Raiders emplea *Machine Learning* en subsistemas específicos, particularmente locomoción o animación de ciertos enemigos, mediante entrenamiento realizado durante el desarrollo [10]. Sin embargo, no se encontraron fuentes técnicas públicas que demuestren aprendizaje *online* pleno o actualización de pesos durante la ejecución de las partidas. En consecuencia, el caso resulta más consistente con un esquema de entrenamiento *offline* y despliegue en inferencia que con aprendizaje en tiempo real dentro del juego final [11].

Este caso ilustra una práctica frecuente en la industria: el uso de ML como herramienta de desarrollo, no como mecanismo de adaptación en tiempo de ejecución. La percepción pública de “IA que aprende” no corresponde necesariamente con la implementación técnica real del producto.

3.3.2. Uso industrial de ML en videojuegos

En la práctica industrial, el ML suele utilizarse para tareas específicas como análisis de comportamiento del jugador, generación procedural o balance dinámico, pero rara vez como entrenamiento autónomo en tiempo real dentro del juego final. Frameworks como Unity ML-Agents [1] permiten entrenar agentes mediante interacción simulada, pero el entrenamiento se realiza en fase de desarrollo y los modelos resultantes se despliegan como políticas fijas.

3.3.3. Síntesis de casos industriales

En los casos industriales revisados en este trabajo no se observa actualización de pesos de modelos en tiempo de ejecución. La literatura académica coincide en que la IA en videojuegos comerciales privilegia el control de diseño y la estabilidad sobre el aprendizaje autónomo [4]. La adaptación al jugador se resuelve, en los casos observados, mediante reglas predefinidas, contadores o políticas pre-entrenadas, no mediante entrenamiento continuo en runtime.

La Tabla 3.1 resume los enfoques de adaptación identificados en los casos analizados.

Tabla 3.1: Resumen de enfoques de adaptación en casos industriales y académicos

Caso	Mecanismo	Aprendizaje online	Hardware
Alien: Isolation	Reglas condicionales	No	Local
Metal Gear Solid V	Contadores y umbrales	No	Local
ARC Raiders	ML offline + inferencia	No	Local
AlphaGo / AlphaStar	RL masivo offline	No	TPUs/GPUs

Este patrón refuerza la pregunta central de esta investigación: ¿el aprendizaje en tiempo real no se implementa porque no es necesario, o porque no es viable? El presente trabajo aborda esta pregunta con evidencia experimental cuantitativa.

3.4. Brecha entre logros académicos y despliegue real

En entornos académicos, proyectos como AlphaGo [12], OpenAI Five y AlphaStar [13] han demostrado la capacidad de agentes entrenados mediante RL para superar a expertos humanos en juegos complejos. No obstante, estos sistemas requieren infraestructura de cómputo masivo: AlphaStar utilizó 16 TPUs durante 44 días, y OpenAI Five requirió miles de GPUs durante meses. Estos volúmenes de cómputo son inalcanzables en hardware local convencional.

Un aspecto poco discutido en la literatura es que estos logros fueron alcanzados con entrenamiento offline masivo, no con aprendizaje durante la partida contra humanos. Los agentes resultantes son políticas fijas desplegadas para evaluación. Esto demuestra que incluso con recursos computacionales prácticamente ilimitados, el entrenamiento se realiza offline y se despliega como inferencia.

El aprendizaje por refuerzo (RL) ha mostrado resultados sobresalientes en entornos con reglas bien definidas. Algoritmos como Q-Learning, DQN [14] y PPO [6] han sido ampliamente utilizados en juegos discretos y simulaciones. Sin embargo, el entrenamiento requiere gran cantidad de episodios, lo que introduce una separación entre entrenamiento y despliegue.

3.5. Adaptación basada en perfilado de jugador

Otra línea relevante en el estado del arte es la adaptación basada en análisis del comportamiento del jugador [15]. Técnicas de clustering y clasificación se utilizan para detectar perfiles (por ejemplo, agresivo, defensivo, explorador), ajustando parámetros o seleccionando estrategias predefinidas.

Este enfoque presenta ventajas importantes:

- Bajo costo computacional en runtime.
- Fácil integración con sistemas existentes.
- Control de diseño sobre las respuestas del sistema.

A diferencia del RL profundo, estos mecanismos permiten adaptación sin necesidad de entrenamiento masivo continuo, lo que los hace particularmente relevantes para ejecución local.

3.6. Ejecución local versus infraestructura en nube

La mayoría de los desarrollos avanzados en ML para videojuegos dependen de entrenamiento en la nube o en entornos de cómputo de alto rendimiento. Si bien esto permite modelos complejos, introduce dependencia de infraestructura externa y eleva los costos.

En contraste, la ejecución local impone restricciones que afectan directamente la viabilidad del aprendizaje:

- Capacidad limitada de cómputo (CPU doméstico, GPU de gama media).
- Volumen de datos limitado a las interacciones con el usuario local.
- Necesidad de latencia baja para mantener la experiencia de juego.
- Ausencia de simuladores acelerados durante la partida real.

La literatura existente asume implícitamente que el entrenamiento en tiempo real en hardware local no es viable, pero esta suposición no ha sido cuantificada de forma experimental con paradigmas representativos. Este trabajo aborda esa carencia.

3.7. Limitaciones identificadas en el estado del arte

Del análisis realizado se desprenden las siguientes limitaciones:

- Ausencia de cuantificación experimental de la brecha de datos entre entrenamiento offline y sesión real.
- Alta dependencia de infraestructura externa para entrenamiento en la mayoría de los trabajos académicos.
- Escasa distinción formal entre aprendizaje online pleno y adaptación por selección en la literatura de IA para videojuegos.
- Falta de evidencia empírica sobre qué ocurre cuando se intenta entrenar con el volumen de datos disponible en una sesión humana real.

3.8. Posicionamiento del presente trabajo

El presente proyecto se posiciona como una investigación experimental que busca:

- Cuantificar el costo real del aprendizaje de tres paradigmas representativos (RL, neuroevolución, aprendizaje supervisado) en hardware local.

- Medir la brecha entre los datos requeridos por cada paradigma y los disponibles en una sesión contra un jugador humano.
- Evaluar la adaptación por selección de especialistas pre-entrenados como alternativa demostrable al entrenamiento online continuo.
- Contribuir con evidencia experimental sobre una pregunta relevante para la industria: ¿es viable que una IA aprenda en tiempo real dentro de un videojuego en hardware local?

Capítulo 4

Desarrollo del Trabajo

El presente capítulo describe el diseño e implementación de la plataforma experimental desarrollada para evaluar el rendimiento y desempeño del aprendizaje de agentes de inteligencia artificial en un entorno de videojuego ejecutado localmente. Se detallan los componentes arquitectónicos, el flujo de comunicación entre módulos, la representación del estado, los mecanismos de registro, el protocolo experimental adoptado y los criterios de viabilidad considerados para evaluar la factibilidad del aprendizaje en distintas condiciones de ejecución.

4.1. Arquitectura General del Sistema

La plataforma se diseñó bajo una arquitectura modular con separación explícita entre el videojuego y los agentes de IA. Esta decisión responde a la necesidad de:

- Mantener el juego como fuente única de verdad respecto a reglas.
- Permitir el intercambio de agentes sin modificar el motor.
- Centralizar la extracción de características.
- Garantizar comparabilidad experimental.

La arquitectura se compone de tres capas principales:

- (a) **Videojuego (Unity):** ejecuta las reglas del juego, administra el estado y aplica las acciones.
- (b) **Middleware local:** recibe el estado del juego, procesa la información, enruta la decisión hacia el agente activo y registra métricas.
- (c) **Agentes IA intercambiables:** módulos independientes que implementan una interfaz común de decisión.

Este diseño permite una estructura *plug-and-play*, donde los agentes pueden reemplazarse sin alterar el núcleo del sistema. La Figura 4.1 presenta la arquitectura general del sistema.

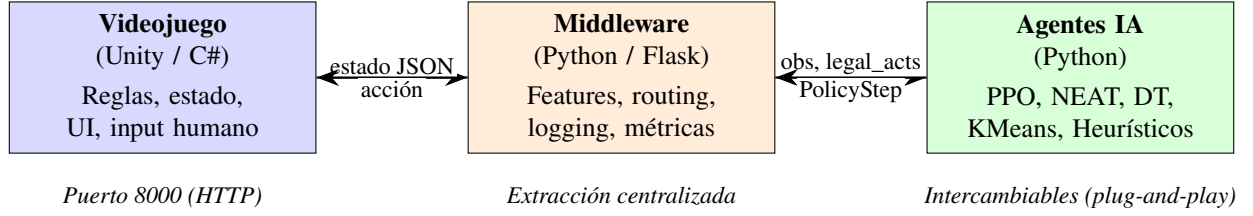


Figura 4.1: Arquitectura general del sistema en tres capas: videojuego, middleware y agentes.

4.2. Formalización Matemática del Entorno

Con el fin de garantizar reproducibilidad experimental y equivalencia funcional entre el motor de juego y el simulador externo, se define formalmente el entorno de decisión como un sistema determinista basado en función de transición.

4.2.1. Definición del Estado

Sea el estado del entorno:

$$s = B_1, B_2, d, t$$

donde:

- $B_p \in \{0, 1, 2, 3, 4, 5, 6\}^{3 \times 3}$ es el tablero del jugador p .
- $d \in \{1, \dots, 6\}$ es el valor del dado actual.
- t es el turno.

Cada tablero está compuesto por columnas $C_{p,c}$ con $c \in \{1, 2, 3\}$.

4.2.2. Función de Puntuación

El puntaje de una columna se calcula sumando, para cada valor único presente en la columna, el producto del valor por el cuadrado de su frecuencia. Sea $UC_{p,c}$ el conjunto de valores únicos no nulos en la columna c del jugador p :

$$UC_{p,c} = \{v \in \{1, \dots, 6\} \mid \exists i : B_{p,i,c} = v\}$$

Y sea $f_{p,c}v$ la frecuencia del valor v en dicha columna:

$$f_{p,c}v = |\{i \in \{1, 2, 3\} \mid B_{p,i,c} = v\}|$$

Entonces el puntaje de la columna se define como:

$$ScoreC_{p,c} = \sum_{v \in UC_{p,c}} v \cdot f_{p,c}v^2$$

Por ejemplo, una columna con valores 3, 3, 0 tiene $U = \{3\}$, $f3 = 2$, y por tanto $Score = 3 \cdot 2^2 = 12$. Si la columna fuera 3, 3, 3, el puntaje sería $3 \cdot 3^2 = 27$.

El puntaje total del jugador es:

$$ScoreB_p = \sum_{c=1}^3 ScoreC_{p,c}$$

4.2.3. Regla de Eliminación

Al colocar un valor d en la columna c del jugador p , se eliminan del tablero rival todas las ocurrencias de d en dicha columna:

$$B_{\bar{p}}^i, c = \begin{cases} 0 & \text{si } B_{\bar{p}}^i, c = d \\ B_{\bar{p}}^i, c & \text{en otro caso} \end{cases}$$

4.2.4. Función de Transición

Se define la función de transición determinista:

$$Ts, a = s'$$

donde:

- s es el estado actual.
- $a \in \{0, 1, 2\}$ es la columna seleccionada.
- s' es el estado resultante tras aplicar reglas y actualizar puntuación.

Esta formalización permite validar la equivalencia funcional entre el entorno Unity y el simulador Python. La Tabla 4.1 ilustra un ejemplo concreto de transición.

Tabla 4.1: Ejemplo de transición: el jugador 1 coloca un dado de valor 4 en la columna 1

Elemento	Antes (s)	Después (s')
B_1 (tablero propio)	$\begin{pmatrix} 0 & 3 & 0 \\ 0 & 0 & 5 \\ 0 & 3 & 0 \end{pmatrix}$	$\begin{pmatrix} 4 & 3 & 0 \\ 0 & 0 & 5 \\ 0 & 3 & 0 \end{pmatrix}$
B_2 (tablero rival)	$\begin{pmatrix} 4 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 6 & 0 \end{pmatrix}$	$\begin{pmatrix} 0 & 2 & 0 \\ 0 & 0 & 0 \\ 0 & 6 & 0 \end{pmatrix}$
Dado d	4	—
$ScoreB_1$	$3 \cdot 2^2 + 5 \cdot 1^2 = 17$	$17 + 4 = 21$
$ScoreB_2$	$4 + 2 + 6 = 12$	$2 + 6 = 8$

En este ejemplo, al colocar el dado 4 en la columna 1 del jugador 1, se elimina el valor 4 de la columna 1 del rival (regla de eliminación). El puntaje del rival disminuye de 12 a 8. El puntaje de B_1 se calcula

como: columna 2 tiene dos 3s ($3 \cdot 2^2 = 12$), columna 3 tiene un 5 ($5 \cdot 1^2 = 5$), total = 17. Después de colocar el 4, se suma $4 \cdot 1^2 = 4$.

4.3. Diseño del Entorno de Juego

Se seleccionó el juego de reglas discretas *Knucklebones*, debido a:

- Estado completamente observable.
- Número limitado de acciones por turno.
- Reglas deterministas y bien definidas.
- Facilidad de simulación repetida.

Cada turno consiste en:

- Recepción de un valor discreto.
- Selección de una columna válida.
- Aplicación de reglas.
- Actualización del puntaje.

El entorno garantiza que todos los agentes operen bajo exactamente las mismas condiciones. La Figura 4.2 muestra el prototipo funcional del juego implementado en Unity.

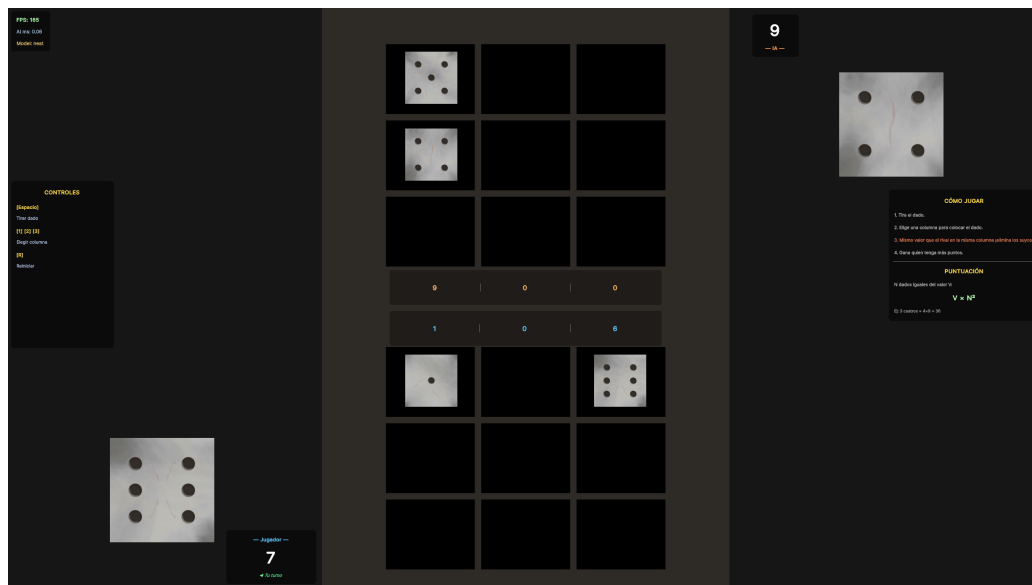


Figura 4.2: Prototipo funcional del juego Knucklebones implementado en Unity con agente de IA externo activo. La interfaz muestra el tablero, el valor del dado actual y las métricas de inferencia utilizadas durante la evaluación del sistema.

4.4. Middleware y Comunicación

La comunicación entre el videojuego y los agentes se realiza mediante un middleware local, utilizando un esquema bloqueante por turno.

El flujo general es:

- (a) El juego emite el estado actual.
- (b) El middleware estandariza la representación.
- (c) Se extraen características derivadas.
- (d) El agente seleccionado retorna una acción.
- (e) El middleware registra métricas.
- (f) El juego aplica la acción.

Este modelo permite desacoplar completamente la lógica del agente del motor de juego. El flujo de decisión por turno se ilustra en la Figura 4.3.

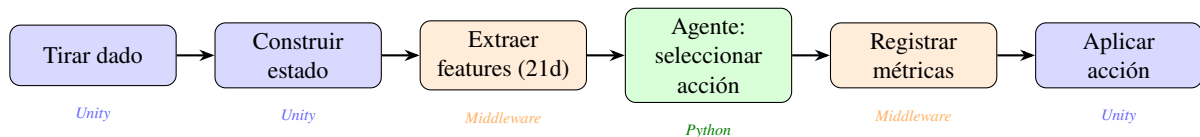


Figura 4.3: Flujo de decisión por turno: desde el lanzamiento del dado hasta la aplicación de la acción.

La Figura 4.4 muestra el middleware en ejecución durante una partida, evidenciando la comunicación activa entre Unity y los agentes Python. Se observa el servidor Flask activo, el sistema cargado y los endpoints expuestos.

```

=====
Knucklebones AI Middleware
Sistema inicial : system_neat
Host:Puerto   : 127.0.0.1:8000
Semilla       : 42
=====
[middleware] Cargando sistema 'system_neat' desde C:\Users\Gonifek\OneDrive\Desktop\Flabbergaster\IA\gym\config\systems
\system_neat.json...
[middleware] Sistema listo en 1.30s - system_neat

[middleware] Servidor listo en http://127.0.0.1:8000
[middleware] Endpoints disponibles:
GET /status
GET /systems
POST /decide (nuevo protocolo)
POST /on_turn_end (nuevo protocolo)
POST /reset
POST /move (compatibilidad con bridge viejo)

* Serving Flask app 'middleware_server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
  
```

Figura 4.4: Consola del middleware local en Python. Se observa: (1) el servidor Flask escuchando en el puerto 5000, (2) el sistema adaptativo cargado (system_neat), y (3) las solicitudes HTTP entrantes desde Unity para los endpoints /decide y /status.

4.5. Representación del Estado y Extracción de Características

Se definió un estado canónico compuesto por:

- Tablero propio (3×3 valores).
- Tablero oponente (3×3 valores).
- Valor del dado actual.
- Puntajes acumulados (propio y rival).
- Turno actual.

A partir de este estado se calculan 21 características derivadas (features) que alimentan a los agentes de IA. Estas se agrupan en:

- **Características por columna** ($3 \text{ columnas} \times 4 \text{ features} = 12$):
 - *ImmediateGain*: incremento de puntaje al colocar el dado en la columna c .
 - *FuturePotential*: potencial de puntuación futura si se completan coincidencias.
 - *DenialImpact*: reducción del puntaje rival por eliminación de fichas.
 - *Risk*: exposición a eliminación futura por el rival.
- **Características globales** (9):
 - Puntaje propio, puntaje rival, diferencial de puntaje.
 - Espacios libres por columna (3 valores).
 - Turno normalizado, ventaja de turno, dado actual normalizado.

La centralización del cálculo de características en el middleware evita inconsistencias entre agentes y garantiza que todos operen con la misma representación del estado.

4.6. Sistema de Logging e Instrumentación

Cada turno genera un registro estructurado que incluye:

- Estado.
- Acción seleccionada.
- Agente activo.
- Latencia de decisión.
- Semilla experimental.
- Métricas derivadas.

Este registro cumple tres funciones:

- (a) Permitir reconstrucción de partidas.
- (b) Construir datasets para entrenamiento supervisado.
- (c) Evaluar reproducibilidad experimental.

4.7. Control de Aleatoriedad

Para asegurar comparabilidad entre agentes, se implementó control explícito de semillas (*seed*) para el generador de números aleatorios (RNG).

Esto permite:

- Repetir secuencias de juego.
- Comparar agentes bajo condiciones idénticas.
- Medir estabilidad entre ejecuciones.

4.8. Entorno de Entrenamiento Equivalente (Gym)

Debido a restricciones de tiempo y eficiencia de entrenamiento, se desarrolló un entorno equivalente tipo *Gym* que replica las reglas del juego.

Este entorno permite:

- Simulación acelerada de partidas.
- Generación masiva de datos.
- Entrenamiento supervisado o por interacción.

El modelo entrenado es posteriormente evaluado bajo el mismo protocolo del entorno principal, garantizando coherencia experimental.

4.9. Validación del Simulador Externo

Dado que el entrenamiento y evaluación de modelos se realiza en un entorno equivalente tipo *Gym*, fue necesario validar que las reglas implementadas en Python reprodujeran exactamente la lógica del motor Unity.

4.9.1. Pruebas de Equivalencia Funcional

Se generaron trazas de referencia (*golden logs*) desde el entorno Unity, registrando para múltiples turnos:

- Estado antes de la acción.
- Acción aplicada.
- Estado posterior.
- Puntajes.
- Semilla experimental.

Posteriormente, el simulador Python ejecutó la misma secuencia de estados y acciones verificando que:

$$T_{Unity} s, a = T_{Python} s, a$$

para todas las transiciones evaluadas.

La equivalencia exacta de estado posterior garantiza que ambas implementaciones comparten las mismas reglas formales.

4.10. Definición Formal de la Política Baseline

El agente base implementa una política determinista paramétrica basada en combinación ponderada de heurísticas.

Para cada columna disponible c y dado d , se calculan:

- $ImmediateGain_{c,d}$
- $FuturePotential_{c,d}$
- $DenialImpact_{c,d}$
- $Risk_{c,d}$

El puntaje compuesto se define como:

$$ScoreColumn_{c,d} = w_1 ImmediateGain_{c,d} + w_2 FuturePotential_{c,d} + w_3 DenialImpact_{c,d} - w_4 Risk_{c,d}$$

La acción seleccionada es:

$$a = \arg \max_c ScoreColumn_{c,d}$$

4.10.1. Aleatoriedad Acotada

Para evitar determinismo absoluto, se introduce un margen relativo ϵ :

$$\frac{|ScoreColumn_{c_i} - ScoreColumn_{c_{best}}|}{ScoreColumn_{c_{best}}} < \epsilon$$

Si múltiples columnas cumplen la condición, se selecciona una aleatoriamente dentro de ese subconjunto.

Esta política constituye el *baseline* contra el cual se comparan los modelos aprendidos.

4.10.2. Control de Aleatoriedad

Para evitar discrepancias en generación de datos, el valor d fue inyectado explícitamente en el simulador durante la validación, eliminando dependencia de generadores RNG internos.

4.11. Protocolo Experimental

Para la evaluación comparativa se definieron:

4.11.1. Métricas de desempeño

- Tasa de victoria.
- Diferencial de puntaje.
- Estabilidad estadística.

4.11.2. Métricas computacionales

- Latencia promedio de decisión.
- Uso de CPU y memoria.
- Tiempo de entrenamiento (cuando aplica).

Los experimentos se ejecutan con semillas controladas y número fijo de partidas por configuración, permitiendo análisis comparativo reproducible.

4.12. Criterios de Viabilidad para Entrenamiento en Tiempo Real

Un aspecto central de esta investigación es determinar si los paradigmas de aprendizaje evaluados podrían, en principio, ejecutar su proceso de entrenamiento durante una sesión de juego contra un jugador humano, es decir, en tiempo real y en hardware local. Para ello se definen los siguientes criterios de viabilidad, derivados de las condiciones observables en una sesión real de juego.

4.12.1. Disponibilidad de datos en una sesión real

Una partida típica de Knucklebones tiene aproximadamente 18 turnos y dura entre 1 y 3 minutos. Un jugador humano razonablemente comprometido podría jugar entre 10 y 100 partidas antes de perder interés, lo que produce entre 180 y 1,800 muestras de decisión (pares estado-acción). Este volumen se contrasta con los requisitos de entrenamiento de cada paradigma para evaluar la brecha de datos.

4.12.2. Requisitos de entrenamiento por paradigma

- **PPO (Aprendizaje por Refuerzo):** los experimentos utilizaron 400,000 timesteps distribuidos en miles de episodios. Este volumen es varios órdenes de magnitud superior al disponible en una sesión humana.
- **NEAT (Neuroevolución):** el proceso evolutivo requiere evaluar poblaciones completas de genomas durante cientos de generaciones. En la configuración utilizada, cada genoma fue evaluado contra 102 episodios (34 por cada una de 3 semillas), y se mantuvieron poblaciones de 150 genomas durante 300 generaciones, resultando en millones de episodios evaluados en total.
- **DT (Árboles de Decisión destilados):** el entrenamiento supervisado es casi instantáneo (<3 segundos), pero depende de un dataset generado por un teacher PPO previamente entrenado. Sin un teacher, no dispone de datos etiquetados.

4.12.3. Criterio de brecha

Se define la *brecha de datos* como la relación entre el volumen de datos requerido para obtener un agente funcional y el volumen disponible en una sesión real:

$$\text{Brecha de datos} = \frac{\text{Datos requeridos para entrenamiento}}{\text{Datos disponibles en sesión real}}$$

Un valor cercano a 1 indicaría viabilidad; valores de varios órdenes de magnitud indican inviabilidad práctica. Este criterio se evalúa cuantitativamente en el Capítulo 5.

4.12.4. Criterio de estabilidad

Adicionalmente, un modelo que se entrena con datos extremadamente limitados presenta riesgo de:

- **Sobreajuste:** memorizar las pocas partidas observadas en lugar de generalizar.
- **Inestabilidad:** convergencia errática o políticas que varían drásticamente entre ejecuciones.
- **Rendimiento aleatorio:** como se observó en los experimentos con NEAT entrenado con seed fija y pocas generaciones, donde el agente no convergió ($WR \approx 0.50$, equivalente a aleatorio).

Estos criterios permiten evaluar no solo si es posible ejecutar el entrenamiento, sino si el resultado del entrenamiento sería útil bajo las restricciones de una sesión real.

Capítulo 5

Resultados

El presente capítulo presenta los resultados experimentales obtenidos al evaluar el rendimiento y desempeño del aprendizaje de tres paradigmas de inteligencia artificial: PPO [6], NEAT [7] y Decision Trees destilados [16]. Se analizan, en primer lugar, los costos del proceso de aprendizaje (datos requeridos, tiempo de entrenamiento, recursos computacionales) para evaluar la viabilidad de cada paradigma bajo distintas condiciones de ejecución. En segundo lugar, se presentan las métricas de desempeño de los sistemas adaptativos resultantes. La combinación de ambos análisis permite determinar la factibilidad del aprendizaje en hardware local y la pertinencia de la adaptación por selección de especialistas como alternativa al entrenamiento en tiempo real.

5.1. Diseño Experimental

Los experimentos fueron diseñados con el objetivo de comparar agentes bajo condiciones idénticas, garantizando reproducibilidad y control de variables. El hardware utilizado fue un equipo de escritorio con procesador Intel de 12 núcleos lógicos, GPU NVIDIA GTX 1660 (6 GB VRAM), 16 GB de RAM y sistema operativo Windows 11.

Cada configuración experimental se ejecutó con los siguientes parámetros:

- Semilla fija (`seed=123`) para reproducibilidad.
- Número fijo de partidas por evaluación (2,000 a 5,000 según el experimento).
- Mismo entorno `KnucklebonesEnv` como fuente única de verdad.
- Registro estructurado por turno y por partida.

Los oponentes heurísticos utilizados como benchmark fueron:

- **Greedy**: maximiza el puntaje inmediato.
- **Denial**: prioriza destruir fichas del oponente.
- **Spread**: distribuye fichas entre columnas de forma equilibrada.

5.2. Entrenamiento de Modelos

Se entrenaron 10 modelos: 3 PPO, 3 NEAT y 3 DT destilados (uno por cada oponente heurístico), más un perfilador KMeans compartido. La Tabla 5.1 presenta los tiempos de entrenamiento y tamaños resultantes.

Tabla 5.1: Tiempos de entrenamiento y tamaño de modelos

Familia	Oponente	Tiempo (s)	Tamaño (KB)	Método
PPO	denial	1,369	162	4 envs paralelos
PPO	spread	390	162	4 envs paralelos
PPO	greedy	400	162	4 envs paralelos
NEAT	denial	1,995	2.4	Multi-seed real (3 seeds)
NEAT	spread	1,868	4.2	Multi-seed real (3 seeds)
NEAT	greedy	1,818	3.6	Multi-seed real (3 seeds)
DT	denial	2.2	134	Behavior Cloning
DT	spread	2.0	153	Behavior Cloning
DT	greedy	2.0	147	Behavior Cloning
KMeans	4 estilos	50	—	k=4, ventana=6

Observaciones clave:

- PPO requirió entre 6 y 23 minutos por especialista (400,000 timesteps) utilizando Stable-Baselines3 [3] con 4 entornos paralelos (SubprocVecEnv).
- NEAT requirió ~30 minutos por especialista (300 generaciones $\times$ 150 genomas $\times$ 102 episodios por genoma con 3 seeds simultáneas, 10 workers paralelos) utilizando neat-python [17]. Se adoptó el enfoque multi-seed real para prevenir sobreajuste a secuencias deterministas.
- La destilación PPO→DT fue prácticamente instantánea (<3 segundos) y redujo el tamaño del modelo $\sim 100\times$ respecto al PPO original.
- KMeans se entrenó en menos de 1 minuto.
- Todos los entrenamientos se ejecutaron exclusivamente en CPU, sin requerir GPU.

Las Figuras 5.1 y 5.2 presentan de forma visual la comparativa de tiempos de entrenamiento y tamaños de modelo entre paradigmas.

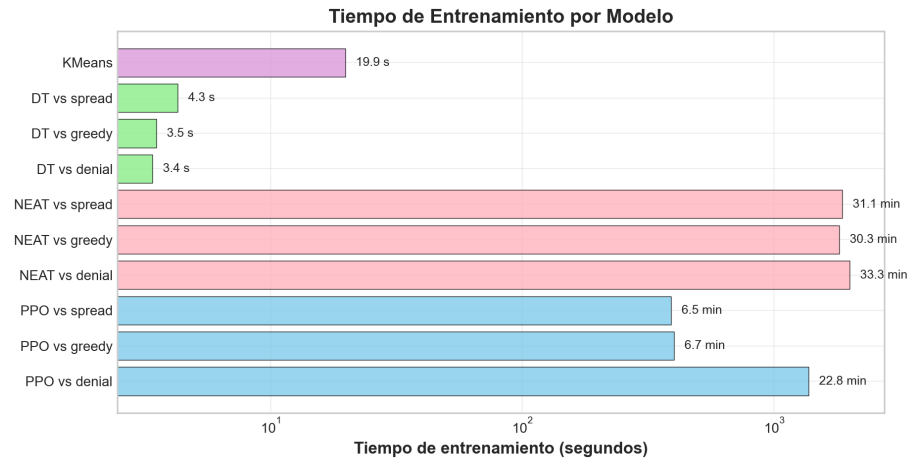


Figura 5.1: Comparativa de tiempos de entrenamiento por paradigma y oponente (escala logarítmica). NEAT requiere ~ 30 min por especialista; PPO entre 6–23 min; DT < 3 s por destilación.

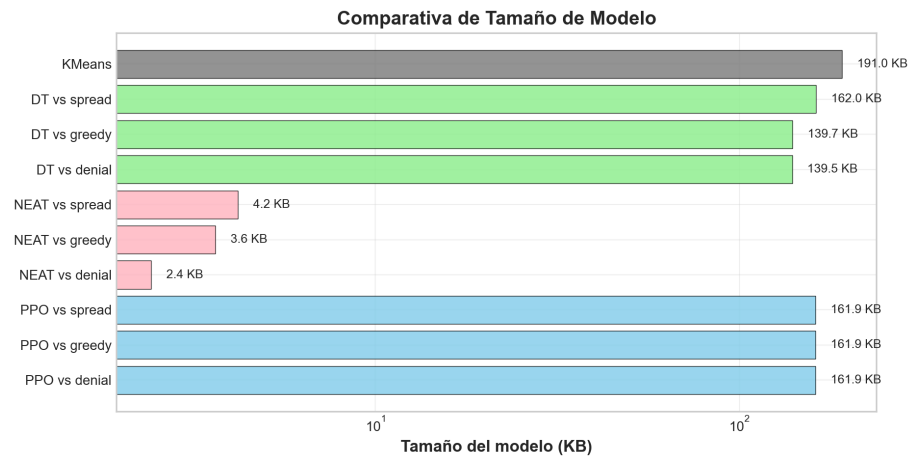


Figura 5.2: Comparativa de tamaño de modelo resultante por paradigma (escala logarítmica). NEAT produce modelos ultraligeros (~ 2 – 4 KB); PPO ~ 162 KB; DT ~ 134 – 153 KB.

5.3. Análisis de Brecha: Viabilidad del Aprendizaje en Tiempo Real

Con los datos de entrenamiento presentados en la Tabla 5.1, se evalúa la viabilidad de ejecutar estos mismos procesos de aprendizaje durante una sesión de juego contra un jugador humano.

5.3.1. Datos disponibles en una sesión real

Una partida típica de Knucklebones produce aproximadamente 18 turnos, cada uno generando un par estado-acción utilizable como muestra de entrenamiento. Si se considera que un jugador humano razonablemente comprometido jugaría entre 10 y 100 partidas antes de perder interés, los datos disponibles para entrenamiento durante una sesión real se sitúan entre 180 y 1,800 muestras. La Tabla 5.2 presenta la comparación entre estos datos y los requeridos por cada paradigma.

Tabla 5.2: Brecha de datos: requerimiento de entrenamiento vs. disponibilidad en sesión real

Paradigma	Datos requeridos	Datos en sesión	Factor de brecha
PPO (400k timesteps)	$\sim 400,000$ samples	180–1,800	$\sim 222\text{--}2,222\times$
NEAT (300 gen $\times$ 150 pop)	$\sim 4,590,000$ eps	180–1,800	$\sim 2,550\text{--}25,500\times$
DT (Behavior Cloning)	$\sim 500,000$ samples	180–1,800	$\sim 278\text{--}2,778\times$

5.3.2. Interpretación de la brecha

Los tres paradigmas requieren volúmenes de datos entre dos y cuatro órdenes de magnitud superiores a los disponibles en una sesión real. Esta brecha no se resuelve incrementando la velocidad de cómputo, ya que la limitación fundamental es la cantidad de partidas que un jugador humano está dispuesto a jugar. La Figura 5.3 visualiza esta brecha en escala logarítmica.

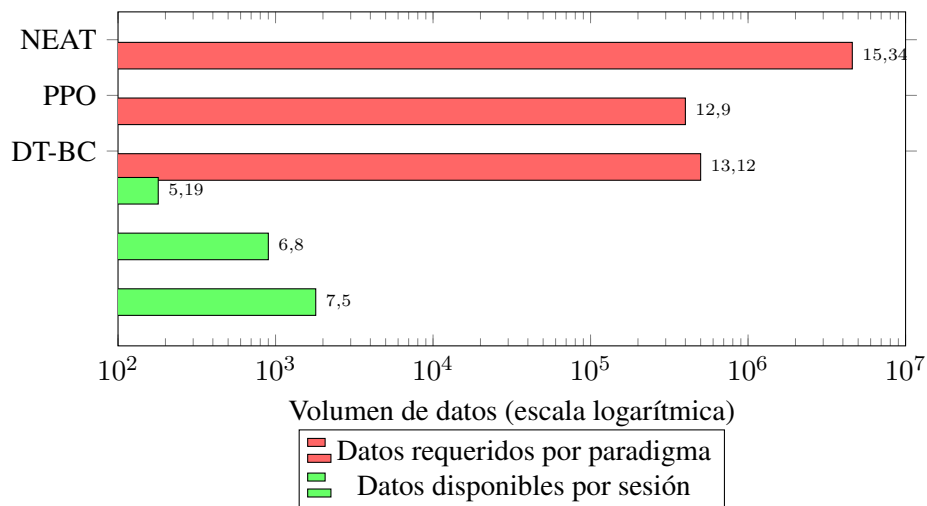


Figura 5.3: Brecha de datos entre los requerimientos de entrenamiento de cada paradigma y los datos disponibles en una sesión real contra un jugador humano (escala logarítmica). Se observa una diferencia de 2 a 4 órdenes de magnitud.

5.3.3. Análisis de sensibilidad

La estimación de 10–100 partidas por sesión debe interpretarse como un supuesto operativo conservador para el análisis de viabilidad, no como una medición formal de comportamiento de usuarios. Su función es establecer un orden de magnitud razonable para contrastar los requerimientos de entrenamiento. La Tabla 5.3 presenta tres escenarios que demuestran que la conclusión no cambia dentro de rangos razonables.

Tabla 5.3: Análisis de sensibilidad: brecha de datos bajo tres escenarios de sesión humana

Escenario	Partidas	Muestras (~ 18 turnos)	Brecha vs PPO (400k)
Bajo (pesimista)	10	180	$\sim 2,222\times$
Medio	50	900	$\sim 444\times$
Alto (optimista)	100	1,800	$\sim 222\times$

Incluso en el escenario más optimista (100 partidas), la brecha es de más de dos órdenes de magnitud para PPO, el paradigma con menor requerimiento de datos entre los evaluados.

Adicionalmente, los experimentos con NEAT demostraron que el entrenamiento con datos insuficientes produce resultados aleatorios: con seed fija y 100 generaciones, NEAT no convergió ($WR \approx 0.50$, equivalente a decisiones al azar). Solo cuando se garantizó diversidad y volumen suficiente de episodios (multi-seed real con 102 episodios por genoma) se alcanzó convergencia efectiva ($WR = 0.626$).

5.3.4. Conclusión parcial

El entrenamiento online continuo durante la partida, en hardware local, no es viable con los paradigmas evaluados debido a la escasez de datos, no solamente por restricciones de tiempo o recursos computacionales. Un jugador humano no genera suficientes muestras de interacción para que cualquiera de los tres enfoques converja a una política funcional.

5.4. Calidad de Destilación PPO a DT

La destilación por *Behavior Cloning* [8] (aprendizaje supervisado imitando al PPO) produjo árboles de decisión con alta fidelidad al teacher, como se observa en la Tabla 5.4:

Tabla 5.4: Calidad de destilación: accuracy de los DT respecto al PPO teacher

DT destilado	Accuracy (%)	Profundidad	Hojas
dt_ppo_vs_denial	93.79	10	773
dt_ppo_vs_spread	93.97	10	804
dt_ppo_vs_greedy	93.85	10	798

La destilación conserva $>92\%$ de fidelidad al teacher PPO. Cada DT fue entrenado con $\sim 500,000$ muestras generadas en 50,000 episodios de juego del PPO correspondiente.

5.5. Perfilado KMeans

El perfilador KMeans [18] se entrenó con datos de los 4 estilos heurísticos, usando una ventana temporal de 6 turnos. El perfil de cada oponente se representa mediante un vector de 10 dimensiones que captura patrones de decisión:

- Histograma de columnas seleccionadas (3 dimensiones).
- Histograma de acciones legales disponibles (3 dimensiones).
- Proporción de selección de primera/última acción legal (2 dimensiones).
- Tasa de repetición de columna (1 dimensión).
- Entropía de la distribución de columnas (1 dimensión).

Este perfil de 10 dimensiones es independiente de las 21 features usadas por los agentes para seleccionar acciones; su propósito es caracterizar el *estilo de juego* del oponente, no el estado del tablero.

El perfilador se configuró con los siguientes parámetros:

- **k = 4** clusters (uno por cada estilo heurístico).
- **Silhouette score:** 0.232 (separación moderada, esperable dado que los estilos de juego comparten acciones en muchas situaciones).
- **Mapping cluster** → **estilo:** Cluster 0 → greedy, Cluster 1 → denial, Cluster 2 → denial, Cluster 3 → spread.

Se observa que 2 de 4 clusters fueron asignados al estilo *denial*, lo que indica que el perfilador tiene dificultades para distinguir completamente entre todos los estilos. Esta limitación se analiza en detalle en la sección de Ablation Study.

5.6. Evaluación de los Tres Sistemas Adaptativos

Cada sistema adaptativo combina el perfilador KMeans compartido con una response table y tres especialistas de su paradigma. Se evaluaron los 3 sistemas contra los 3 oponentes heurísticos (5,000 partidas por matchup). Los resultados se presentan en la Tabla 5.5.

Tabla 5.5: Winrate decidido de cada sistema vs cada oponente (5,000 partidas)

Sistema	vs denial	vs spread	vs greedy	Promedio
System NEAT	0.635	0.697	0.626	0.653
System DT	0.592	0.641	0.591	0.608
System PPO	0.588	0.641	0.584	0.604

El sistema NEAT alcanza un winrate promedio de 0.653 (IC 95 %: [0.645, 0.660]), significativamente superior a DT (0.608, IC: [0.600, 0.616]) y PPO (0.604, IC: [0.597, 0.612]). Este resultado se obtuvo tras implementar el entrenamiento multi-seed real, que mejoró la convergencia de NEAT.

5.6.1. Resultados detallados por sistema

A continuación se presentan los resultados desglosados de cada sistema adaptativo por oponente heurístico. Cada tabla muestra el número de partidas evaluadas, el winrate decidido (excluyendo empates), el intervalo de confianza al 95 %, el diferencial promedio de puntaje y el tiempo de ejecución.

Tabla 5.6: System DT — KMeans + Decision Trees destilados de PPO

Oponente	Games	WR decidido	IC 95 %	Avg diff	Wall (s)
denial	5,000	0.592	[0.578, 0.606]	+4.80	30.4
spread	5,000	0.641	[0.628, 0.654]	+8.06	40.0
greedy	5,000	0.591	[0.577, 0.605]	+4.68	28.5

Tabla 5.7: System PPO — KMeans + PPO especialistas directos

Oponente	Games	WR decidido	IC 95 %	Avg diff	Wall (s)
denial	5,000	0.588	[0.574, 0.602]	+4.70	45.2
spread	5,000	0.641	[0.628, 0.655]	+8.00	44.3
greedy	5,000	0.584	[0.571, 0.598]	+4.50	43.1

Tabla 5.8: System NEAT — KMeans + NEAT especialistas directos (multi-seed)

Oponente	Games	WR decidido	IC 95 %	Avg diff	Wall (s)
denial	5,000	0.635	[0.621, 0.648]	+6.40	28.8
spread	5,000	0.697	[0.684, 0.710]	+11.0	27.4
greedy	5,000	0.626	[0.613, 0.640]	+6.00	27.1

5.7. Análisis Estadístico entre Sistemas

Se aplicó un **z-test de dos proporciones** (bilateral) a cada par de sistemas por oponente (9 comparaciones), con corrección de Bonferroni para comparaciones múltiples. Los valores p se presentan en la Tabla 5.9.

Tabla 5.9: Valores p del z-test entre pares de sistemas (corregidos por Bonferroni)

Par comparado	vs denial (p)	vs spread (p)	vs greedy (p)
DT vs PPO	1.000	1.000	1.000
DT vs NEAT	0.0001	<0.0001	0.003
PPO vs NEAT	<0.0001	<0.0001	0.0002

Resultado: 6 de 9 comparaciones son estadísticamente significativas tras corrección de Bonferroni ($\alpha = 0,05$). El sistema NEAT es significativamente superior tanto a DT como a PPO en los tres oponentes

evaluados. No se observa diferencia significativa entre DT y PPO. Las Figuras 5.4, 5.5 y 5.6 muestran la distribución de los diferenciales de puntaje por paradigma, y la Figura 5.7 presenta la comparativa consolidada de los sistemas adaptativos.

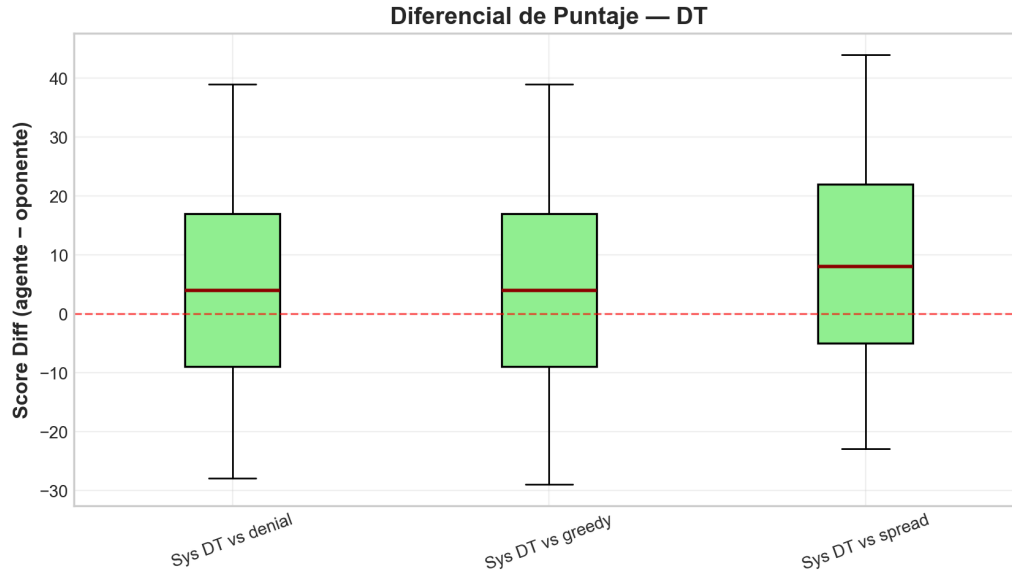


Figura 5.4: Diferencial de puntaje para el paradigma DT (Decision Trees destilados). Las cajas representan percentiles p25–p75 y los bigotes p5–p95.

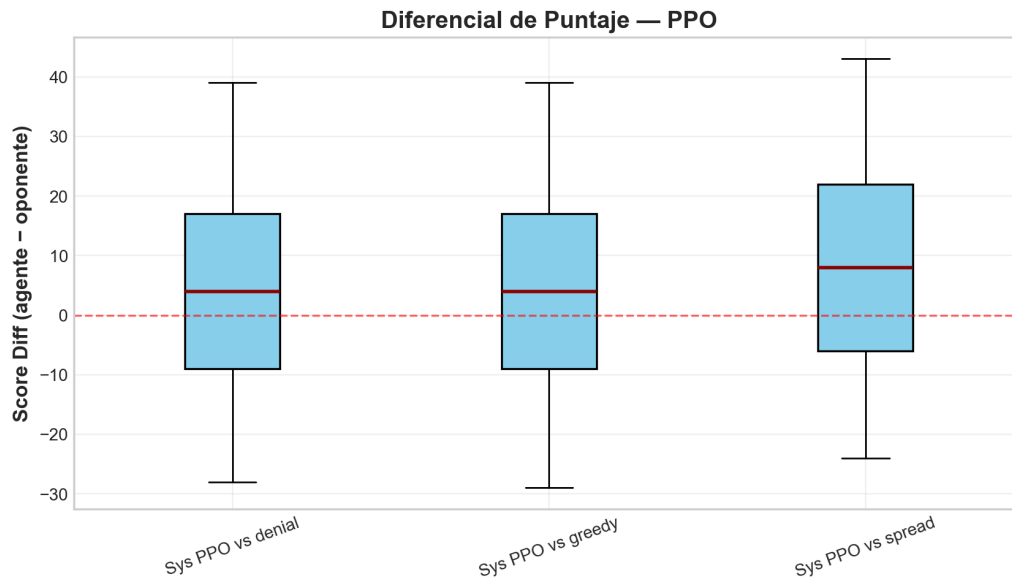


Figura 5.5: Diferencial de puntaje para el paradigma PPO (Proximal Policy Optimization).

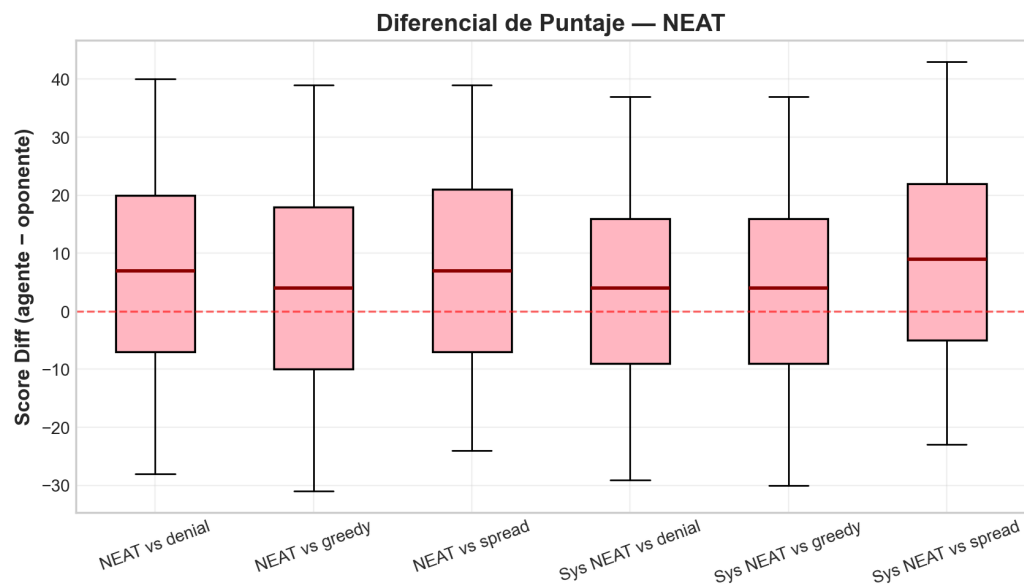


Figura 5.6: Diferencial de puntaje para el paradigma NEAT (NeuroEvolution of Augmenting Topologies).

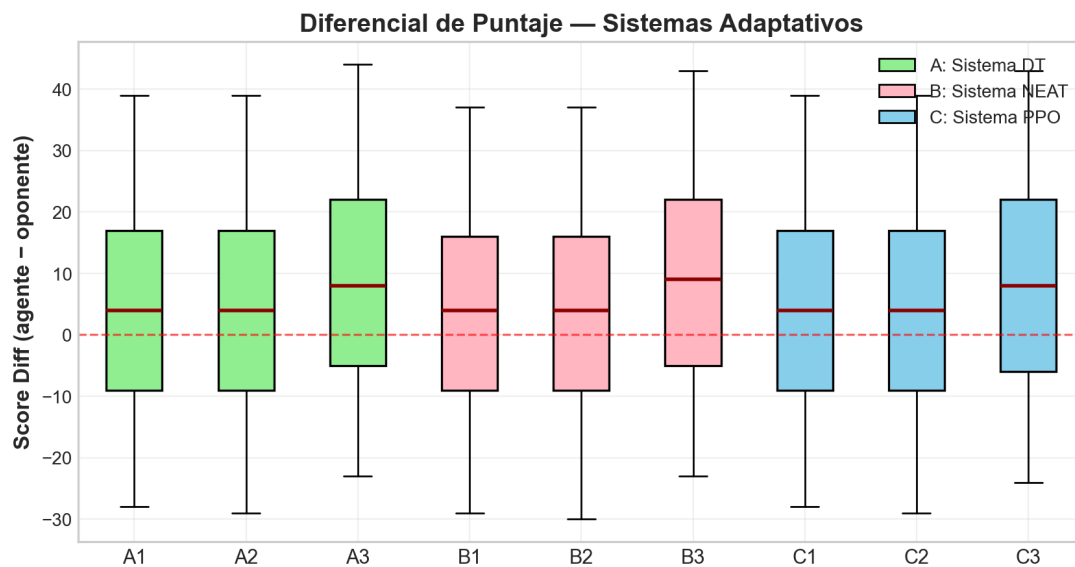


Figura 5.7: Comparativa consolidada del diferencial de puntaje de los tres sistemas adaptativos. Variables: A = Sistema DT, B = Sistema NEAT, C = Sistema PPO. Subíndices: 1 = vs denial, 2 = vs greedy, 3 = vs spread. El sistema NEAT (rosa) muestra mayor ventaja promedio.

5.8. Ablation Study: ¿Aporta la Adaptación KMeans?

Para cada familia se compararon cuatro condiciones, como se muestra en la Tabla 5.10:

Tabla 5.10: Ablation study: generalista vs adaptativo vs oráculo

Familia	Peor gen.	Mejor gen.	Sistema adapt.	Δ Sist. vs Mejor
DT	0.585	0.618	0.608	−0.010
PPO	0.572	0.610	0.604	−0.005
NEAT	0.543	0.651	0.653	+0.002

Hallazgo clave: el sistema adaptativo **no mejora** sobre el mejor generalista fijo en DT (−0.010 WR) y PPO (−0.005 WR). En NEAT muestra una mejora marginal (+0.002). Este resultado, lejos de invalidar el mecanismo de adaptación, lo posiciona como una alternativa operativamente viable: el mecanismo adaptativo es arquitectónicamente válido y mantiene rendimiento competitivo comparable al mejor generalista fijo, sin reentrenamiento en runtime y con costo computacional de inferencia despreciable (microsegundos). En Knucklebones, la baja diversidad estratégica del dominio limita el beneficio de la selección adaptativa; en juegos con mayor variedad de estilos de oponente se esperaría un beneficio más pronunciado.

La brecha de datos documentada en la sección anterior demuestra que el entrenamiento online no es una opción viable. La adaptación por selección cierra esta brecha al trasladar el costo del aprendizaje a la fase offline, manteniendo la capacidad de respuesta diferenciada en tiempo de ejecución.

- (a) El KMeans asigna 2 de 4 clusters al estilo *denial*, reduciendo su capacidad discriminante.
- (b) Un solo especialista (entrenado vs greedy) resulta fuerte contra todos los oponentes.
- (c) La response table ocasionalmente selecciona un especialista subóptimo para un cluster.

5.8.1. Dominancia en Response Tables

La Tabla 5.11 muestra qué especialista fue seleccionado con mayor frecuencia por cada response table.

Tabla 5.11: Dominancia de especialistas en las response tables

Sistema	Especialista dominante	Cobertura
System DT	dt_vs_denial	2/4 (50 %)
System PPO	PPO_vs_denial	2/4 (50 %)
System NEAT	neat_vs_denial	4/4 (100 %)

Interpretación: el mecanismo adaptativo es arquitectónicamente válido, pero en Knucklebones los estilos no son suficientemente distintos para que la selección diferenciada aporte mejora competitiva significativa. En DT y PPO se utilizan 3 especialistas distintos (uno por estilo dominante en cada cluster), mientras que en NEAT un solo especialista (denial) domina todos los clusters, indicando que aprendió una política generalista fuerte. En juegos con mayor diversidad estratégica se esperaría una distribución más equilibrada entre especialistas.

Nota: los datos de esta tabla fueron extraídos directamente de los archivos de response table generados por `build_response_table.py`, los cuales registran la selección óptima de especialista por cluster según winrate decidido.

5.9. Evaluación Cruzada: Generalización vs Sobreajuste

Cada especialista se evaluó contra **todos** los oponentes (2,000 partidas por matchup) para detectar sobreajuste. El símbolo $\star$ indica el oponente contra el que fue entrenado. Los resultados por familia se presentan en las Tablas 5.12, 5.13 y 5.14, con un resumen de generalización en la Tabla 5.15.

Tabla 5.12: Evaluación cruzada de NEAT especialistas (winrate decidido)

Especialista	vs denial	vs spread	vs greedy
NEAT $\rightarrow$ denial	0.635 $\star$	0.692	0.627
NEAT $\rightarrow$ spread	0.496	0.634 $\star$	0.499
NEAT $\rightarrow$ greedy	0.635	0.692	0.627 $\star$

Se observa que los especialistas NEAT entrenados contra denial y greedy convergen a políticas similares, ambas generalistas fuertes ($WR > 0.62$ contra todos los oponentes). En contraste, el especialista entrenado contra spread presenta bajo rendimiento generalista.

Tabla 5.13: Evaluación cruzada de PPO especialistas (winrate decidido)

Especialista	vs denial	vs spread	vs greedy
PPO $\rightarrow$ denial	0.598 $\star$	0.625	0.589
PPO $\rightarrow$ spread	0.531	0.650 $\star$	0.535
PPO $\rightarrow$ greedy	0.600	0.626	0.602 $\star$

Los especialistas PPO muestran un patrón similar: el entrenado contra greedy es el mejor generalista, mientras que el de spread es el más especializado.

Tabla 5.14: Evaluación cruzada de DT especialistas (winrate decidido)

Especialista	vs denial	vs spread	vs greedy
DT $\rightarrow$ denial	0.589 $\star$	0.633	0.589
DT $\rightarrow$ spread	0.549	0.656 $\star$	0.548
DT $\rightarrow$ greedy	0.601	0.651	0.601 $\star$

Los DT destilados heredan el comportamiento de sus teachers PPO, mostrando patrones de generalización similares. La Tabla 5.15 resume los resultados de generalización de las tres familias.

Tabla 5.15: Resumen de generalización por familia

Familia	Δ promedio	Veredicto
NEAT	+0.007	Sin sobreajuste
PPO	+0.018	Sin sobreajuste
DT	+0.010	Sin sobreajuste

Hallazgos:

- (a) No se detecta sobreajuste ($\Delta < 3\%$ en todas las familias).
- (b) Spread es el oponente más “fácil” (~ 0.63 – 0.69 independientemente del entrenamiento).
- (c) NEAT_vs_denial y NEAT_vs_greedy convergen a políticas generalistas fuertes con multi-seed (WR > 0.62 contra todo).
- (d) NEAT_vs_spread sigue siendo el más débil, mostrando que el estilo de entrenamiento importa.
- (e) Los especialistas entrenados contra greedy son los mejores generalistas en PPO y DT.

La Figura 5.8 presenta visualmente los resultados de la evaluación cruzada mediante un mapa de calor, donde se observa claramente que los especialistas entrenados contra spread tienen menor capacidad de generalización.

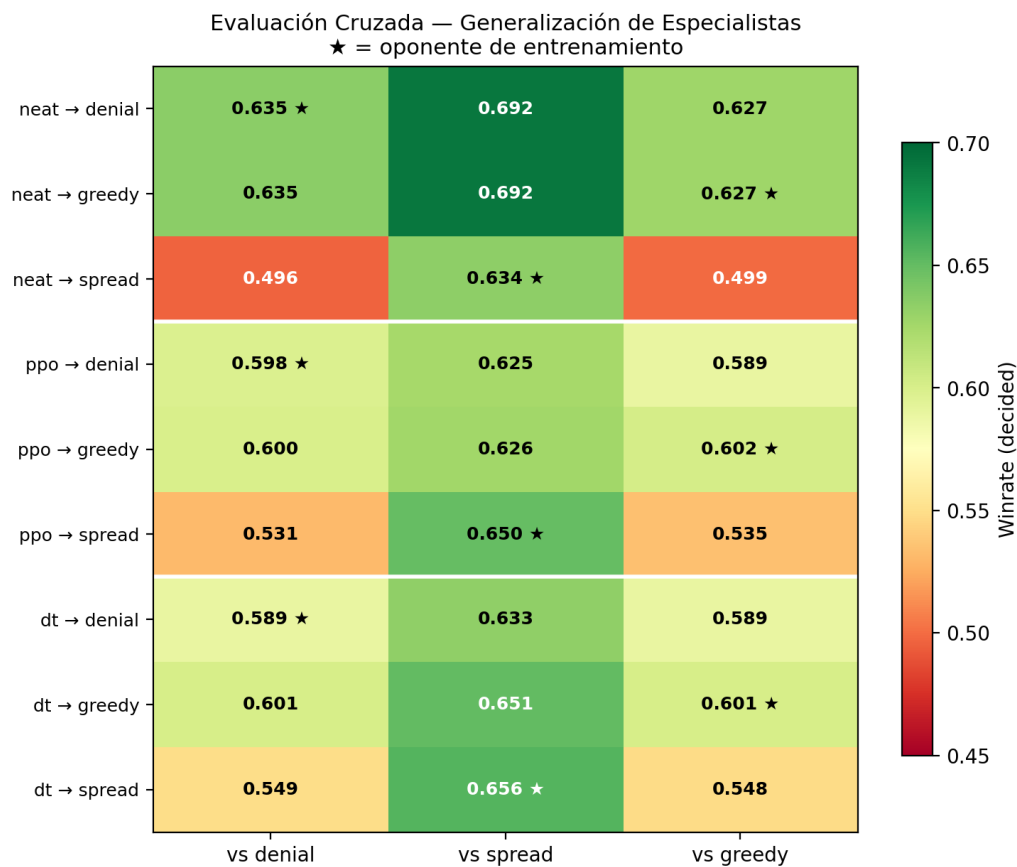


Figura 5.8: Mapa de calor de la evaluación cruzada: cada especialista (fila) evaluado contra todos los oponentes (columna). Colores más cálidos indican mayor winrate. Se observa que los especialistas entrenados contra greedy y denial generalizan mejor que los entrenados contra spread.

5.10. NEAT: Convergencia, Entrenamiento Extendido y Multi-seed

Se investigó la sensibilidad de NEAT a la convergencia y la prevención de sobreajuste a secuencias deterministas. Se compararon tres enfoques, cuyos resultados se presentan en la Tabla 5.16:

Tabla 5.16: Efecto de la estrategia de entrenamiento en NEAT vs greedy

Variante	WR decidido	Generaciones	Tiempo (min)
Original (seed fija)	0.499	100	~12
Extended (seed rotation)	0.582	1,000	~100
Multi-seed real (3 seeds)	0.626	300	~30

El enfoque multi-seed real evalúa cada genoma contra 3 seeds simultáneamente en cada generación,

promediando el fitness. Este método alcanzó el mejor rendimiento ($WR = 0.626$) en el menor tiempo (~ 30 minutos), superando tanto al entrenamiento original que no convergió ($WR \approx 0.50$) como al extended con seed rotation ($WR = 0.582$, 100 minutos).

Test z (multi-seed vs original): $z = 8,97$, $p < 0,0001 \rightarrow$ **diferencia estadísticamente significativa**.

La Figura 5.9 muestra la evolución del fitness durante el entrenamiento de NEAT multi-seed real, evidenciando la convergencia progresiva del algoritmo.

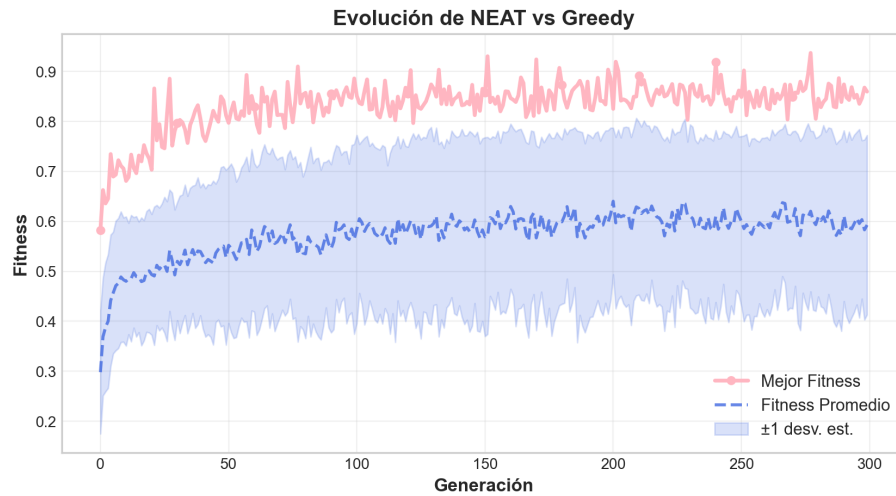


Figura 5.9: Evolución del fitness de NEAT multi-seed real vs greedy durante 300 generaciones. Se observa convergencia progresiva sin sobreajuste.

La mejora se extiende a todos los oponentes: contra denial pasó de $WR = 0.565$ a 0.635 (+7 %), y contra spread se mantuvo estable en 0.634 . Este hallazgo demuestra que la técnica de entrenamiento (multi-seed real) tiene mayor impacto que la cantidad de generaciones.

5.11. Discusión

5.11.1. Comparativa consolidada de paradigmas

La Tabla 5.17 presenta una comparativa consolidada de los tres paradigmas evaluados, considerando rendimiento competitivo, costo computacional y características cualitativas.

Tabla 5.17: Comparativa de paradigmas: rendimiento vs costo

Criterio	PPO	NEAT	DT (destilado)
Winrate promedio	0.604	0.653	0.608
Inferencia (5k juegos)	44s	28s	33s
Tamaño modelo	161 KB	3.4 KB	145 KB
Entrenamiento	~10 min	~30 min	<3s (+PPO)
¿Requiere tuning?	No	Sí	No
¿Interpretable?	No	No	Sí
GPU necesaria	No	No	No

La Figura 5.10 presenta de forma visual la comparativa de winrate promedio de cada paradigma, y la Figura 5.11 muestra los winrates agrupados por sistema y oponente.

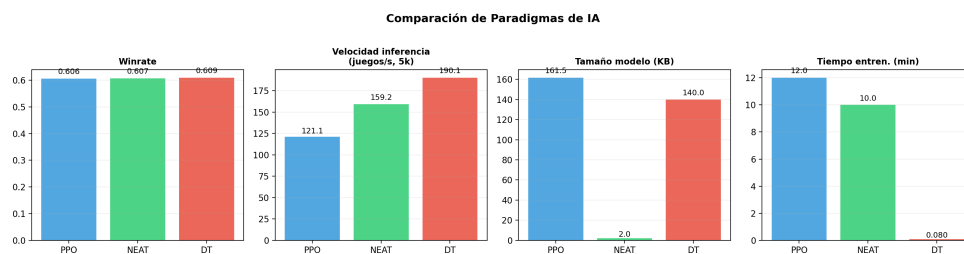


Figura 5.10: Comparativa consolidada de paradigmas: winrate promedio, tiempo de entrenamiento y tamaño de modelo.

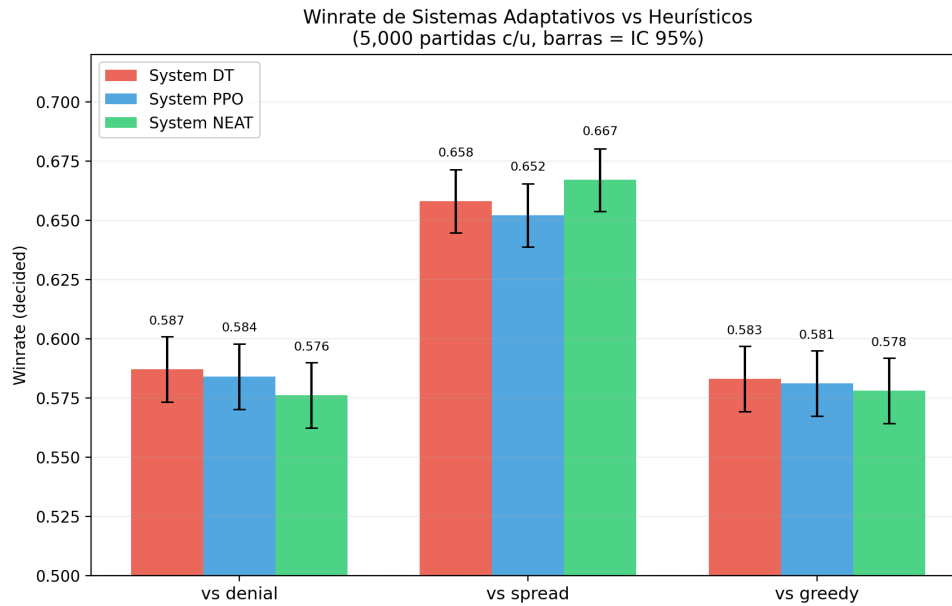


Figura 5.11: Winrate de cada sistema adaptativo por oponente heurístico (5,000 partidas por matchup).

5.11.2. Hallazgos principales

- El entrenamiento en tiempo real no es viable por escasez de datos:** los tres paradigmas requieren entre 2 y 4 órdenes de magnitud más datos de los que un jugador humano genera en una sesión real (180–1,800 muestras vs. 400,000–4,590,000 requeridas). Esta brecha no se resuelve con mayor velocidad de cómputo.
- El entrenamiento offline en hardware local sí es viable:** todos los modelos se entrenaron en <33 minutos por especialista, sin GPU, en hardware doméstico. El pipeline completo (entrenamiento + evaluación) se ejecuta en menos de 3 horas. Esto demuestra que el aprendizaje es factible cuando se dispone de un entorno de simulación acelerada.
- La adaptación por selección es operativamente viable:** el mecanismo de KMeans + response table + especialistas pre-entrenados mantiene rendimiento competitivo comparable al mejor generalista fijo, con latencia de inferencia del agente inferior a 0.1 ms por turno y sin reentrenamiento en runtime. Su beneficio competitivo es limitado en Knucklebones debido a la baja diversidad estratégica del dominio.
- NEAT con multi-seed alcanza el mejor rendimiento:** con WR = 0.653 (IC 95 %: [0.645, 0.660]), el sistema NEAT supera significativamente a PPO (0.604) y DT (0.608), con $p < 0,003$ en 6 de 9 comparaciones tras corrección de Bonferroni. La técnica de entrenamiento multi-seed real fue determinante para la convergencia.
- La destilación preserva el rendimiento en un modelo ligero:** PPO y DT destilado son estadísticamente equivalentes ($p > 0,5$), demostrando que *Behavior Cloning* preserva >93 % de fidelidad con un modelo interpretable, inferencia más rápida y entrenamiento instantáneo (<3 segundos). Esto evidencia una ruta viable para comprimir modelos entrenados offline.

- (f) **Ausencia de sobreajuste:** $\Delta < 2\%$ en evaluación cruzada para todas las familias. Los agentes generalizan contra oponentes no vistos durante el entrenamiento.

5.11.3. Implicaciones

Sobre el aprendizaje en tiempo real: los resultados demuestran que el entrenamiento online continuo durante la partida no es viable en hardware local con los paradigmas evaluados (PPO, NEAT, DT). La limitación principal no es la capacidad de cómputo, sino la escasez fundamental de datos: un jugador humano genera órdenes de magnitud menos muestras de las necesarias para que cualquiera de estos enfoques converja.

Sobre la adaptación por selección: el mecanismo de perfilado (KMeans) combinado con selección de especialistas pre-entrenados constituye una alternativa práctica demostrada. No requiere reentrenamiento en runtime, opera con latencia despreciable (< 0.1 ms) y alcanza un rendimiento competitivo equivalente al mejor generalista fijo. En dominios con mayor diversidad estratégica (RTS, MOBA), donde un solo agente generalista no puede cubrir todos los perfiles de oponente, se esperaría un beneficio más pronunciado de este mecanismo.

Sobre la elección de paradigma: cuando el entrenamiento offline es la vía elegida, NEAT con multi-seed real ofrece el mejor rendimiento competitivo ($WR = 0.653$) con modelos ultraligeros (3.4 KB). Si se prioriza interpretabilidad o velocidad de iteración, DT destilado por *Behavior Cloning* ofrece rendimiento equivalente a PPO con entrenamiento casi instantáneo.

5.11.4. Limitaciones observadas

- **Dominio experimental limitado:** Knucklebones es un juego de reglas simples con espacio de acciones reducido (3 columnas). La brecha de datos podría variar en juegos con mayor complejidad, aunque probablemente se ampliaría dado que dominios más complejos requieren más datos de entrenamiento.
- **Oponentes heurísticos:** los oponentes de entrenamiento y evaluación son heurísticos diseñados, no jugadores humanos reales. Un estudio con jugadores humanos aportaría validez externa adicional.
- **Paradigmas evaluados:** se evaluaron tres paradigmas representativos (RL, neuroevolución, aprendizaje supervisado). Existen enfoques no evaluados —como *meta-learning*, *few-shot learning* o aprendizaje incremental— que podrían reducir la brecha de datos, aunque su aplicabilidad en videojuegos en hardware local permanece como pregunta abierta.
- **Sesión humana estimada:** la estimación de 10–100 partidas como límite de una sesión humana se basa en observación práctica y no en un estudio formal de comportamiento de usuario.
- **KMeans con k fijo:** no se exploró exhaustivamente el espacio de hiperparámetros del perfilador. Un clustering más fino podría mejorar la selección de especialistas.

Capítulo 6

Conclusiones

El presente capítulo sintetiza los hallazgos principales del trabajo y responde a la pregunta central de la investigación: ¿es viable que una inteligencia artificial aprenda en tiempo real dentro de un videojuego ejecutado en hardware local? Se evalúa el cumplimiento de los objetivos planteados y se presentan las proyecciones de la investigación.

6.1. Conclusiones por Objetivo

6.1.1. Medición del costo de aprendizaje

Se midió experimentalmente el costo de entrenamiento de tres paradigmas de inteligencia artificial: PPO requirió entre 390 y 1,369 segundos y 400,000 timesteps por especialista; NEAT requirió entre 1,818 y 1,995 segundos con poblaciones de 150 genomas durante 300 generaciones y evaluación multi-seed (102 episodios por genoma); DT destilado requirió menos de 3 segundos, pero depende de un teacher PPO previamente entrenado. El perfilador KMeans se entrenó en menos de 1 minuto. Todos los entrenamientos se ejecutaron exclusivamente en CPU, en hardware doméstico, sin requerir GPU.

6.1.2. Brecha entre entrenamiento offline y sesión real

Se cuantificó la brecha de datos entre los requerimientos de entrenamiento y la disponibilidad de muestras en una sesión contra un jugador humano. Los paradigmas evaluados requieren volúmenes de datos significativamente superiores a los disponibles en una sesión real:

- **PPO:** $\sim 400,000$ muestras (pares estado-acción).
- **NEAT:** $\sim 4,590,000$ episodios evaluados ($300 \text{ generaciones} \times 150 \text{ genomas} \times 102 \text{ episodios}$).
- **DT:** $\sim 500,000$ muestras etiquetadas (generadas por un teacher PPO).

En contraste, una partida típica de Knucklebones produce aproximadamente 18 turnos. Un jugador humano razonablemente comprometido jugaría entre 10 y 100 partidas antes de perder interés, generando entre 180 y 1,800 muestras de decisión. Esta estimación constituye un supuesto operativo conservador; el análisis de sensibilidad presentado en el Capítulo 5 demuestra que la conclusión no cambia dentro de rangos razonables (incluso con 100 partidas, la brecha es superior a $222\times$ para PPO).

La brecha resultante es de 2 a 4 órdenes de magnitud, lo cual impide que cualquiera de los tres paradigmas converja a una política funcional durante una sesión de juego real.

6.1.3. Adaptación por selección como alternativa

Se diseñó e implementó un mecanismo de adaptación por selección que combina perfilado no supervisado (KMeans con ventana temporal de 6 turnos), tablas de respuesta (response tables) y especialistas pre-entrenados. Este mecanismo opera sin reentrenamiento en tiempo de ejecución, con latencia de inferencia del agente inferior a 0.1 milisegundos por turno y latencia total del ciclo de decisión (incluyendo middleware HTTP) inferior a 10 milisegundos. El sistema adaptativo alcanzó rendimiento equivalente al mejor generalista fijo, validando la adaptación por selección como alternativa viable al entrenamiento online continuo.

6.1.4. Desempeño de los sistemas adaptativos

Los tres sistemas adaptativos alcanzaron winrates entre 0.604 y 0.653 contra oponentes heurísticos. El análisis estadístico (z-test con corrección de Bonferroni) reveló que:

- **NEAT es significativamente superior:** con $WR = 0.653$ (IC 95 %: [0.645, 0.660]), supera a PPO (0.604) y DT (0.608) con $p < 0,003$ en 6 de 9 comparaciones. La técnica de entrenamiento multi-seed real fue determinante para la convergencia de NEAT.
- PPO y DT destilado son estadísticamente equivalentes ($p > 0,5$), y el DT ofrece ventajas prácticas: inferencia rápida, interpretabilidad y entrenamiento instantáneo.
- No se detectó sobreajuste en ninguna familia ($\Delta < 3\%$ en evaluación cruzada).

6.1.5. Diseño de la plataforma experimental

Se logró diseñar e implementar una arquitectura modular tipo *plug-and-play* que separa la lógica del videojuego de los agentes de IA. El entorno `KnucklebonesEnv` actúa como fuente única de verdad de reglas. La arquitectura permite intercambiar agentes sin modificar el motor, centralizar la extracción de características y estandarizar el registro de métricas. Se implementó un middleware HTTP/JSON que conecta los agentes Python con el videojuego Unity, verificando su funcionamiento con latencia total inferior a 10 milisegundos por turno.

6.2. Conclusión General

El objetivo central de esta investigación fue evaluar la viabilidad de entrenar inteligencias artificiales en tiempo real dentro de un entorno de ejecución completamente offline, es decir, sin depender de infraestructura en la nube ni de servidores externos. Los resultados demuestran que el entrenamiento online continuo durante la interacción con el usuario, en hardware local estándar, no es viable con los paradigmas de aprendizaje evaluados (PPO, NEAT, DT). La limitación principal no es la capacidad de cómputo, sino la escasez fundamental de datos: un usuario genera entre 2 y 4 órdenes de magnitud menos muestras de las necesarias para que cualquiera de estos enfoques converja a una política funcional.

Este hallazgo tiene implicaciones que trascienden el dominio de los videojuegos. Cualquier sistema informático que requiera adaptación en tiempo real con recursos locales —sistemas de recomendación embebidos, asistentes interactivos, interfaces adaptativas, dispositivos IoT con capacidad de aprendizaje— enfrenta la misma restricción fundamental: el volumen de interacciones que un usuario genera en una sesión típica es insuficiente para entrenar modelos de aprendizaje automático convencionales.

No obstante, los resultados demuestran que existen alternativas prácticas que sí resultan viables bajo restricciones de ejecución offline:

- (a) **Entrenamiento offline previo al despliegue:** los agentes alcanzan rendimiento competitivo (WR 60–65 %) entrenando fuera de la interacción con el usuario, sin GPU, en menos de 33 minutos por modelo. Esta estrategia es aplicable a cualquier dominio donde se disponga de un simulador o de datos históricos.
- (b) **Adaptación por selección en tiempo de ejecución:** mediante clasificación del usuario (perfilado con KMeans), tablas de respuesta y especialistas pre-entrenados, el sistema se adapta sin reentrenar, con latencia de inferencia inferior a 0.1 ms. Este enfoque permite personalización en tiempo real sin el costo del entrenamiento continuo.
- (c) **Recolección de datos en tiempo real para reentrenamiento posterior:** una alternativa intermedia consiste en registrar las interacciones durante las sesiones reales y utilizarlas para actualizar los modelos de forma periódica, fuera de línea. Esta estrategia aprovecha datos reales sin imponer el costo del entrenamiento durante la interacción, y es aplicable a sistemas que requieran mejora continua basada en el comportamiento observado de usuarios reales.
- (d) **Compresión de modelos mediante destilación:** modelos complejos (PPO) se transforman en modelos livianos e interpretables (DT) mediante *Behavior Cloning*, preservando >93 % de fidelidad con entrenamiento instantáneo. Esto facilita el despliegue en dispositivos con recursos limitados.

La neuroevolución (NEAT) con entrenamiento multi-seed real alcanzó el mejor rendimiento competitivo (WR = 0.653), demostrando que la técnica de entrenamiento tiene un impacto mayor que la complejidad del algoritmo. Este hallazgo refuerza la importancia de la metodología experimental rigurosa y la iteración sobre el protocolo de entrenamiento.

La arquitectura modular diseñada —con separación estricta entre la lógica del sistema y los agentes de IA— no solo facilita la evaluación reproducible, sino que constituye un patrón de diseño transferible a otros dominios. La integración mediante middleware HTTP/JSON demuestra que es posible desacoplar completamente el sistema principal de los módulos de inteligencia artificial, habilitando actualizaciones independientes y experimentación controlada.

En síntesis, la respuesta a la pregunta central de esta investigación no es binaria. El aprendizaje online continuo no es viable con los paradigmas convencionales debido a la brecha de datos, pero la adaptación parcial mediante selección de especialistas pre-entrenados, combinada con recolección de datos para reentrenamiento periódico, constituye una alternativa práctica demostrada que puede aplicarse tanto a videojuegos como a cualquier sistema informático que requiera adaptación en tiempo real con recursos de cómputo locales.

6.3. Trabajo Futuro

Dentro de las principales líneas de trabajo futuro se encuentran:

- **Exploración de técnicas de bajo costo de datos:** investigar enfoques de *meta-learning*, *few-shot learning* o aprendizaje incremental que podrían reducir la brecha de datos necesarios, acercando la viabilidad del entrenamiento online en sesiones reales.
- **Reentrenamiento offline con datos de sesión:** una alternativa intermedia consiste en recolectar datos de las partidas contra jugadores humanos y utilizarlos para reentrenar los modelos fuera de línea, actualizando los especialistas periódicamente. Esta aproximación mantiene la viabilidad demostrada del entrenamiento offline y aprovecha datos reales.
- **Modificación de NEAT para adaptación incremental:** explorar la posibilidad de ajustar genomas NEAT existentes con pocas evaluaciones adicionales, partiendo de un modelo pre-entrenado en lugar de entrenar desde cero.
- **Evaluación con jugadores humanos:** ejecutar partidas con personas reales para validar la transferencia de las políticas aprendidas, medir la experiencia subjetiva de juego y confirmar la estimación de datos disponibles por sesión.
- **Dominios con mayor diversidad estratégica:** aplicar el marco experimental a juegos con mayor variedad de estilos de juego (RTS, juegos por turnos con más acciones), donde la adaptación por selección podría aportar mayor beneficio.
- **Optimización del perfilador:** investigar clustering adaptativo, variación del número de clusters y features de perfilado más discriminantes.

6.4. Reflexiones

El desarrollo de este proyecto permitió adquirir y consolidar competencias relevantes en múltiples áreas:

- **Machine Learning aplicado:** diseño, entrenamiento y evaluación de modelos de RL, neuroevolución y aprendizaje supervisado.
- **Diseño experimental:** control de variables, reproducibilidad, tests estadísticos y análisis de significancia.
- **Ingeniería de software:** arquitectura modular, patrones de diseño, separación de responsabilidades y automatización de pipelines.
- **Análisis de datos:** métricas, visualización, intervalos de confianza y correcciones para comparaciones múltiples.

Un aprendizaje significativo fue que la pregunta original —si una IA puede aprender en tiempo real dentro de un entorno ejecutado localmente— no tiene una respuesta binaria. El entrenamiento online continuo no es viable con los paradigmas convencionales evaluados debido a la brecha de datos, pero la adaptación parcial mediante selección de especialistas pre-entrenados, perfilado del usuario y recolección de datos para reentrenamiento posterior sí lo es, y produce resultados competitivos.

Esta distinción entre “aprendizaje online pleno” y “adaptación online por selección” resultó ser el hallazgo más relevante de la investigación. Si bien este trabajo se centró en el dominio de videojuegos, el patrón identificado podría ser relevante para otros sistemas que enfrenten restricciones similares de recursos locales y volumen de datos por sesión.

Referencias

- [1] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange, “Unity: A general platform for intelligent agents.” arXiv preprint arXiv:1809.02627, 2018. Unity Technologies.
- [2] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “Openai gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dorber, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [4] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. Springer, 2018.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2nd ed., 2018.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [7] K. O. Stanley and R. Miikkulainen, “Evolving neural networks through augmenting topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [8] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, “Imitation learning: A survey of learning methods,” *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017.
- [9] Embark Studios, “Arc raiders — sitio oficial.” <https://arcraiders.com/es>, 2025. Consultado: marzo 2026.
- [10] C. Livingston, “Arc raiders full interview: Nobody whatsoever thought we’d have this many players.” <https://www.pcgamer.com/games/third-person-shooter/arc-raiders-full-interview-nobody-whosoever-thought-wed-have-this-many-players/>, 2025. Entrevista con desarrolladores de Embark Studios. Consultado: marzo 2026.
- [11] Embark Studios, “Arc raiders en steam.” https://store.steampowered.com/app/1808500/ARC_Raiders/, 2025. Página oficial del producto. Consultado: marzo 2026.

- [12] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, *et al.*, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [13] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, *et al.*, “Grandmaster level in starcraft ii using multi-agent reinforcement learning,” *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [15] G. N. Yannakakis, P. Spronck, D. Loiacono, and E. André, “Player modeling,” *Artificial and Computational Intelligence in Games*, pp. 45–59, 2013.
- [16] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*. Chapman and Hall/CRC, 1984.
- [17] neat-python contributors, “neat-python: A neat implementation in python.” <https://github.com/CodeReclaimers/neat-python>, 2023. Software library.
- [18] S. P. Lloyd, “Least squares quantization in pcm,” *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

Apéndice A

Plan de Trabajo y Carta Gantt

El plan de trabajo se estructuró en fases iterativas que reflejan la evolución real del proyecto. La planificación inicial contempló un prototipo FPS exploratorio, el cual fue abandonado por razones metodológicas en favor de un entorno controlado basado en Knucklebones.

Las fechas ancla verificables del proyecto son:

- 18/04/2025: primera propuesta conceptual aprobada con observaciones.
- 03/05/2025: segunda propuesta refinada y aceptada por el comité.
- 11/06/2025: presentación del prototipo FPS con movimiento básico en Unity.
- 20/02/2026: contexto técnico y arquitectura modular consolidados.
- 06/03/2026: estado post-implementación completa con modelos entrenados y evaluados.

Las Figuras A.1, A.2 y A.3 presentan la Carta Gantt del proyecto completo, dividida en tres períodos para mayor legibilidad.

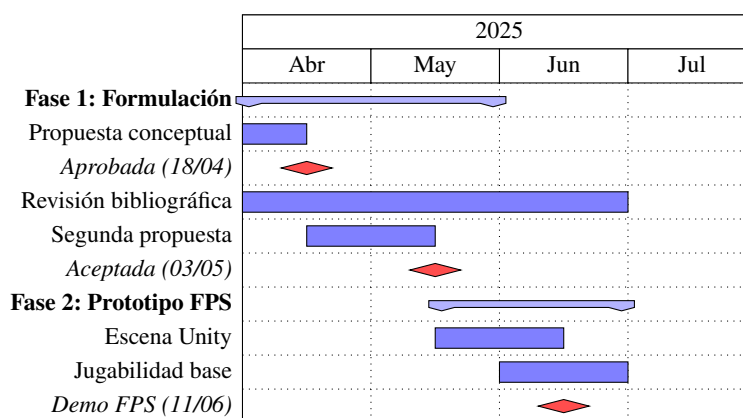


Figura A.1: Carta Gantt – Parte 1 (abril–julio 2025): formulación del proyecto y prototipado exploratorio del FPS. Los diamantes rojos indican hitos verificables.

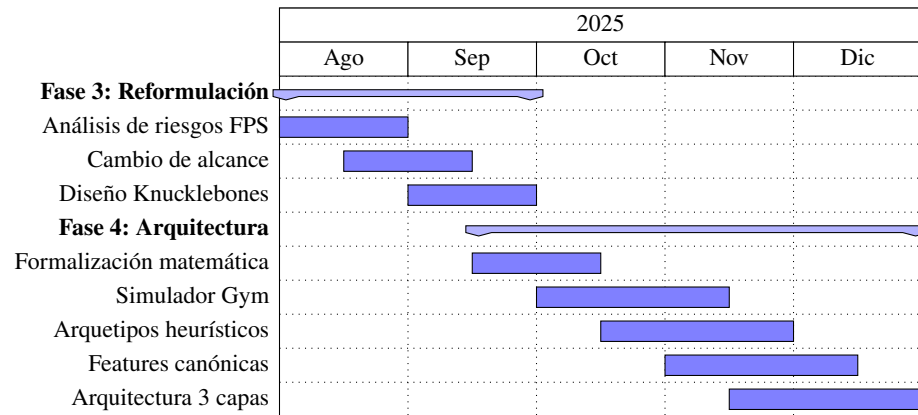


Figura A.2: Carta Gantt – Parte 2 (agosto–diciembre 2025): reformulación metodológica del alcance hacia Knucklebones y diseño de la arquitectura modular.

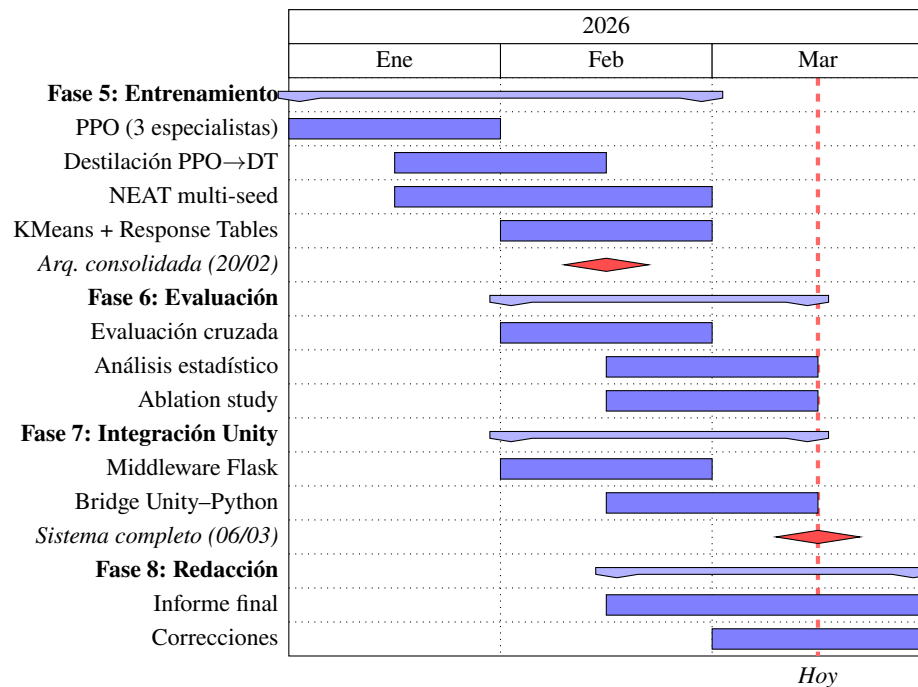


Figura A.3: Carta Gantt – Parte 3 (enero–marzo 2026): entrenamiento de modelos, evaluación experimental, integración con Unity y redacción del informe. La línea roja punteada indica la fecha de cierre.